读享 V1.0 软件

前 30 页

生成日期:2026-06-15

摘选总行数:5103

文件清单:

- person/notfresh/readingshare/MainActivity.java
- person/notfresh/readingshare/ShareUtil.java
- person/notfresh/readingshare/ui/home/HomeFragment.java
- person/notfresh/readingshare/adapter/LinkAdapter.java
- person/notfresh/readingshare/db/LinkDao.java
- person/notfresh/readingshare/db/DatabaseHelper.java
- person/notfresh/readingshare/model/Link.java
- person/notfresh/readingshare/model/Tag.java
- person/notfresh/readingshare/ui/tag/TagFragment.java
- person/notfresh/readingshare/adapter/TagAdapter.java
- person/notfresh/readingshare/ui/subject/SubjectDetailActivity.java
- person/notfresh/readingshare/ui/archive/ArchiveFragment.java
- person/notfresh/readingshare/util/RecentTagsManager.java
- person/notfresh/readingshare/util/StringUtil.java

```
1  package person.notfresh.readingshare;
2
3  import android.annotation.SuppressLint;
4  import android.os.Bundle;
5  import android.view.LayoutInflater;
6  import android.view.View;
7  import android.view.Menu;
8  import android.view.MenuItem;
9  import android.content.Intent;
10 import android.net.Uri;
11
12 import java.util.ArrayList;
13 import java.util.List;
14 import java.util.Map;
15
16 import android.util.Log;
17 import android.content.pm.PackageManager;
18 import android.content.pm.ResolveInfo;
19
20 import com.google.android.flexbox.FlexboxLayout;
21 import com.google.android.material.snackbar.Snackbar;
22 import com.google.android.material.navigation.NavigationView;
23
24 import androidx.navigation.NavController;
25 import androidx.navigation.NavDestination;
26 import androidx.navigation.Navigation;
27 import androidx.navigation.ui.AppBarConfiguration;
28 import androidx.navigation.ui.NavigationUI;
29 import androidx.drawerlayout.widget.DrawerLayout;
30 import androidx.appcompat.app.AppCompatActivity;
31 import androidx.fragment.app.Fragment;
32
33 import person.notfresh.readingshare.databinding.ActivityMainBinding;
34 import person.notfresh.readingshare.db.LinkDao;
35 import person.notfresh.readingshare.model.LinkItem;
36 import person.notfresh.readingshare.ui.home.HomeFragment;
37 import android.content.ClipboardManager;
38 import android.content.ClipData;
39 import android.app.AlertDialog;
40 import android.widget.EditText;
41 import android.text.TextUtils;
42 import java.io.BufferedReader;
43 import java.io.InputStreamReader;
44 import java.net.URL;
45 import java.nio.charset.StandardCharsets;
46 import android.os.Handler;
47 import android.os.Looper;
48 import java.util.concurrent.ExecutorService;
49 import java.util.concurrent.Executors;
50 import java.util.regex.Matcher;
```

```
import java.util.regex.Pattern;
import android.content.Context;
import android.Manifest;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import android.app.ActivityManager;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.net.URLConnection;
import android.content.SharedPreferences;
import androidx.core.view.GravityCompat;
import person.notfresh.readingshare.util.BilibiliUrlConverter;
import person.notfresh.readingshare.util.CrawlUtil;
import person.notfresh.readingshare.util.RecentTagsManager;
import person.notfresh.readingshare.util.StringUtil;

import java.io.IOException;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;
import android.widget.Spinner;
import android.widget.ArrayAdapter;
import java.io.File;
import person.notfresh.readingshare.util.SubjectUtil;
import person.notfresh.readingshare.db.SubjectDao;
import person.notfresh.readingshare.core.model.Subject;

public class MainActivity extends AppCompatActivity {

    private AppBarConfiguration mAppBarConfiguration;
    private ActivityMainBinding binding;
    private LinkDao linkDao;
    private boolean hasFocus = false;
    private String lastClipboardText = "";  // ■■■■■■■■■■■■■■■■■■■■■
    private NavController navController;  // ■ navController ■■■■■■■■

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        try {
            binding = ActivityMainBinding.inflate(getLayoutInflater());
            setContentView(binding.getRoot());

            linkDao = new LinkDao(this);
            linkDao.open();

            setSupportActionBar(binding.appBarMain.toolbar);

            DrawerLayout drawer = binding.drawerLayout;
```

```
101             NavigationView navigationView = binding.navView;
102
103             // ■■■■■■■■
104             navController = Navigation.findNavController(this, R.id.nav_host_fra
105
106             // ■■■■■■■■■DocumentViewerActivity■■■■■■■
107             handleNavigation(getIntent());
108
109             // ■■■■■ AppBarConfiguration
110             // ■■■nav_tags■■■■■■■■■■nav_home
111             mAppBarConfiguration = new AppBarConfiguration.Builder(
112                     R.id.nav_home, R.id.nav_slideshow, R.id.nav_rss, R.id.nav_ar
113                     .setOpenableLayout(drawer)
114                     .build();
115
116             NavigationUI.setupActionBarWithNavController(this, navController, mA
117             NavigationUI.setupWithNavController(navigationView, navController);
118
119             // ■■■■■■■■■■
120             View headerView = navigationView.getHeaderView(0);
121             headerView.setOnClickListener(v -> {
122                 drawer.closeDrawer(GravityCompat.START);
123                 new Handler().postDelayed(() -> {
124                     Intent intent = new Intent(this, UserProfileActivity.class);
125                     startActivity(intent);
126                 }, 250);
127             });
128
129             // ■■■■■■
130             handleIntent(getIntent());
131
132             // ■■■■■■■■■
133             navigationView.setNavigationItemSelectedListener(item -> {
134                 int id = item.getItemId();
135                 drawer.closeDrawer(GravityCompat.START);
136
137                 new Handler().postDelayed(() -> {
138                     try {
139                         if (id == R.id.nav_home) {
140                             navController.navigate(R.id.nav_home);
141                         } else if (id == R.id.nav_rss) {
142                             navController.navigate(R.id.nav_rss);
143                         } else if (id == R.id.nav_slideshow) {
144                             navController.navigate(R.id.nav_slideshow);
145                         } else if (id == R.id.nav_archive) {
146                             navController.navigate(R.id.nav_archive);
147                         } else if (id == R.id.nav_documents) {
148                             navController.navigate(R.id.nav_documents);
149                         } else if (id == R.id.nav_subject) {
150                             navController.navigate(R.id.nav_subject);
```

```
151                         }
152                     } catch (Exception e) {
153                         Log.e("MainActivity", "Navigation failed", e);
154                         Toast.makeText(this, "■■■■", Toast.LENGTH_SHORT).show();
155                     }
156                 }, 250);
157 
158                 return true;
159             });
160 
161             checkClipboardPermission();
162 
163             // ■■FAB■■■■
164             binding.appBarMain.fab.setOnClickListener(view -> {
165                 try {
166                     // ■■■■■■■■■■“■■■”■■■■■■■■“■■■”■■
167                     Intent qqMailIntent = new Intent(Intent.ACTION_SENDTO);
168                     qqMailIntent.setPackage("com.tencent.androidqqmail");
169                     qqMailIntent.setClassName(
170                             "com.tencent.androidqqmail",
171                             "com.tencent.qqmail.launcher.third.LaunchComposeMail
172                     );
173                     qqMailIntent.setData(Uri.parse("mailto:notfresh@foxmail.com"
174                     qqMailIntent.putExtra(Intent.EXTRA_SUBJECT, "■■■■");
175                     qqMailIntent.putExtra(Intent.EXTRA_TEXT, "■■■■■■■■■■■■...");
176 
177                     try {
178                         startActivity(qqMailIntent);
179                     } catch (Exception e) {
180                         // ■■■■■■■■■■
181                         Intent emailIntent = new Intent(Intent.ACTION_SENDTO);
182                         emailIntent.setData(Uri.parse("mailto:notfresh@foxmail.c
183                         emailIntent.putExtra(Intent.EXTRA_SUBJECT, "■■■■");
184                         emailIntent.putExtra(Intent.EXTRA_TEXT, "■■■■■■■■■■■■...
185                         startActivity(emailIntent);
186                     }
187                 } catch (Exception e) {
188                     Log.e("MainActivity", "Error opening email app", e);
189                     Toast.makeText(this, "■■■■■■■■", Toast.LENGTH_SHORT).show();
190                 }
191             });
192 
193         } catch (Exception e) {
194             Log.e("MainActivity", "onCreate failed", e);
195             Toast.makeText(this, "■■■■■■: " + e.getMessage(), Toast.LENGTH_LONG)
196             finish();
197         }
198     }
199 
200     @Override
```

```
201      public void onWindowFocusChanged(boolean hasFocus) { //@mark
202          super.onWindowFocusChanged(hasFocus);
203          this.hasFocus = hasFocus;
204
205          if (hasFocus) {
206              // ■■■■■■■■■■■■■■■■
207              checkClipboard(); //@Def Line300
208          }
209      }
210
211      @Override
212      public boolean onCreateOptionsMenu(Menu menu) {
213          // ■ Fragment ■■■■
214          return super.onCreateOptionsMenu(menu);
215      }
216
217      @Override
218      protected void onResume() {
219          super.onResume();
220          updateNavHeader();
221      }
222
223      @Override
224      protected void onDestroy() {
225          super.onDestroy();
226          if (linkDao != null) {
227              linkDao.close();
228          }
229      }
230
231      @Override
232      public boolean onOptionsItemSelected(MenuItem item) {
233          // ■■ Fragment ■■■■■■■
234          Fragment currentFragment = getSupportFragmentManager().findFragmentById(
235          if (currentFragment != null && currentFragment.onOptionsItemSelected(ite
236              return true;
237          }
238          return super.onOptionsItemSelected(item);
239      }
240
241      @Override
242      public boolean onSupportNavigateUp() {
243          return NavigationUI.navigateUp(navController, mAppBarConfiguration)
244                  || super.onSupportNavigateUp();
245      }
246
247
248      @Override
249      protected void onNewIntent(Intent intent) {
250          super.onNewIntent(intent);
```

```
251          setIntent(intent);
252          // ■■■■■■■DocumentViewerActivity■■■■■
253          if (navController != null) {
254              handleNavigation(intent);
255          }
256          handleIntent(intent);
257      }
258
259      /**
260       * ■■■■■■■
261       */
262      private void handleNavigation(Intent intent) {
263          if (intent == null || navController == null) {
264              return;
265          }
266
267          String navigateTo = intent.getStringExtra("navigate_to");
268          int destinationId;
269
270          if ("documents".equals(navigateTo)) {
271              // ■■■■■■PDF■■■■■■■■■■■■■
272              destinationId = R.id.nav_documents;
273          } else if ("home".equals(navigateTo)) {
274              // ■■■■■■WebView■■■■■■■■■■
275              destinationId = R.id.nav_home;
276          } else {
277              // ■■■■■Tab■■■■■■
278              SharedPreferences prefs = getPreferences(Context.MODE_PRIVATE);
279              int defaultTab = prefs.getInt("default_tab", 0); // ■■■0■■■■■
280
281              // ■■■■■■■■■■■ID
282              switch (defaultTab) {
283                  case 0: // ■■
284                      destinationId = R.id.nav_home;
285                      break;
286                  case 2: // ■■
287                      destinationId = R.id.nav_subject;
288                      break;
289                  case 3: // RSS
290                      destinationId = R.id.nav_rss;
291                      break;
292                  case 4: { // ■■■■■■/■■/RSS
293                      int r = (int) (Math.random() * 3);
294                      destinationId = (r == 0)
295                          ? R.id.nav_home
296                          : (r == 1 ? R.id.nav_subject : R.id.nav_rss);
297                      break;
298                  }
299                  default: // ■■■■■■■■■■■case 1■■■■■
300                      destinationId = R.id.nav_home;
```

```
301                     break;
302             }
303         }
304
305         // ■■■■■■■
306         try {
307             navController.navigate(destinationId);
308         } catch (Exception e) {
309             Log.e("MainActivity", "■■■■", e);
310         }
311     }
312
313     private void handleIntent(Intent intent) {
314         String action = intent.getAction();
315         String type = intent.getType();
316
317         if (Intent.ACTION_SEND.equals(action) && type != null) {
318             if ("text/plain".equals(type)) {
319                 String sharedText = intent.getStringExtra(Intent.EXTRA_TEXT);
320                 String title = intent.getStringExtra(Intent.EXTRA_SUBJECT);
321
322                 // ■■Intent■■■■■■■
323                 Log.d("ShareIntent", "Action: " + action);
324                 Log.d("ShareIntent", "Type: " + type);
325                 Log.d("ShareIntent", "Text: " + sharedText);
326                 Log.d("ShareIntent", "Subject: " + title);
327                 if (intent.getComponent() != null) {
328                     Log.d("ShareIntent", "Package: " + intent.getComponent().get
329                 }
330                 Log.d("ShareIntent", "Data: " + intent.getDataString());
331                 Bundle extras = intent.getExtras();
332                 if (extras != null) {
333                     for (String key : extras.keySet()) {
334                         Log.d("ShareIntent", "Extra: " + key + " = " + extras.ge
335                     }
336                 }
337
338                 // ■■■■■■Intent■■
339                 Log.d("ShareIntent", "=== ■■■■■■ ===");
340                 Log.d("ShareIntent", "Intent: " + intent.toString());
341                 Log.d("ShareIntent", "Categories: " + intent.getCategories());
342                 Log.d("ShareIntent", "Flags: " + Integer.toHexString(intent.getF
343                 Log.d("ShareIntent", "Scheme: " + intent.getScheme());
344
345                 if (intent.getComponent() != null) {
346                     Log.d("ShareIntent", "Component: " + intent.getComponent().f
347                 }
348
349                 // ■■■■■■■■■■Intent■■■
350                 PackageManager pm = getPackageManager();
```

```
351                 List<ResolveInfo> activities = pm.queryIntentActivities(intent,
352                 for (ResolveInfo info : activities) {
353                     Log.d("ShareIntent", "■■■■■■■: " + info.activityInfo.packageN
354                     Log.d("ShareIntent", "Activity: " + info.activityInfo.name);
355                 }
356 
357                 if (sharedText != null) {
358                     // ■■■■■■■■■■■■
359                     String sourceApp = "";
360                     if (intent.getComponent() != null) {
361                         sourceApp = intent.getComponent().getPackageName();
362                     }
363 
364                     // ■■■■■
365                     String initialTitle = (title == null || title.isEmpty())
366                             ? StringUtil.extractTitle(sharedText) : title;
367 
368                     // ■■■■■ Activity ■■■■■■■■■■
369                     StringBuilder targetActivityInfo = new StringBuilder();
370                     for (ResolveInfo info : activities) {
371                         targetActivityInfo.append(info.activityInfo.packageName)
372                                 .append("/")
373                                 .append(info.activityInfo.name)
374                                 .append(";");
375                     }
376 
377                     // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
378                     showSaveLinkDialog(
379                             sharedText,
380                             initialTitle,
381                             sourceApp,
382                             intent.toString(),
383                             targetActivityInfo.toString(),
384                             false
385                     );
386                 }
387             }
388         }
389     }
390 
391     private void checkClipboardPermission() {
392         // ■■■■■■■■■■■■■■■■
393         checkClipboard();
394     }
395 
396     private void checkClipboard() {
397         try {
398             // ■■■■■■■■■■■■■■■
399             if (!hasFocus || !isAppInForeground()) {
400                 Log.d("Clipboard", "App is not in focus or foreground");
```

```
                return;
            }

            // ■■■■■■■■■■■■■■■■■■
            new Handler(Looper.getMainLooper()).postDelayed(() -> {
                try {
                    Log.d("Clipboard", "Checking clipboard...");
                    ClipboardManager clipboard = (ClipboardManager) getSystemSer
                    if (clipboard == null || !clipboard.hasPrimaryClip()) {
                        return;
                    }
                    ClipData clipData = clipboard.getPrimaryClip();
                    if (clipData == null || clipData.getItemCount() == 0) {
                        return;
                    }
                    CharSequence clipTextSequence = clipData.getItemAt(0).getTex
                    if (clipTextSequence == null) {
                        return;
                    }
                    SharedPreferences prefs = getSharedPreferences("clipboard_pr
                    String savedText = prefs.getString("last_clipboard_text", ""
                    if (!clipTextSequence.toString().equals(savedText)) {
                        lastClipboardText = clipTextSequence.toString();
                        handleClipboardText(lastClipboardText); //@Def Line376
                    }
                    prefs.edit().putString("last_clipboard_text", clipTextSequen

                } catch (Exception e) {
                    Log.e("Clipboard", "Error checking clipboard", e);
                    e.printStackTrace();
                }
            }, 500);

        } catch (Exception e) {
            Log.e("Clipboard", "Error in checkClipboard", e);
            e.printStackTrace();
        }
    }

    private boolean isAppInForeground() {
        ActivityManager activityManager = (ActivityManager) getSystemService(Con
        List<ActivityManager.RunningAppProcessInfo> appProcesses = activityManag
        if (appProcesses == null) return false;

        String packageName = getPackageName();
        for (ActivityManager.RunningAppProcessInfo appProcess : appProcesses) {
            if (appProcess.importance == ActivityManager.RunningAppProcessInfo.I
                    && appProcess.processName.equals(packageName)) {
                return true;
            }
```

```
451          }
452          return false;
453      }
454
455      private void handleClipboardText(String text) {
456          if (TextUtils.isEmpty(text)) {
457              return;
458          }
459
460          Log.d("LinkParser", "Processing text: " + text);
461
462          // ■■■■■ http(s) URL
463          Pattern urlPattern = Pattern.compile("https?://[^\\s,■]+");
464          Matcher urlMatcher = urlPattern.matcher(text);
465
466          if (urlMatcher.find()) {
467              // ■■URL
468              String url = urlMatcher.group(0);
469              // ■■URL■■■■■■■■
470              String cleanUrl = url.split("\\?")[0];
471              // ■■■■■.xml■■■■■■■■■
472              if (cleanUrl.toLowerCase().endsWith(".xml")) {
473                  return;
474              }
475
476              // ■■URL■■■■■■■■■
477              String title = text.substring(0, urlMatcher.start()).trim();
478
479              // ■■■■■■■■■■■■■■■
480              if (TextUtils.isEmpty(title)) {
481                  title = "■■■";
482              } else if (title.length() > 20) {
483                  title = title.substring(0, 20) + "...";
484              }
485
486              Log.d("LinkParser", "Extracted title: " + title);
487              Log.d("LinkParser", "Extracted URL: " + url);
488
489              // ■■■■■■■
490              final String finalTitle = title;
491              final String finalUrl = url;
492              new Handler(Looper.getMainLooper()).post(() -> {
493                  showSaveLinkDialog(finalUrl, finalTitle);
494              });
495          }
496      }
497
498      /** ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
499      private void showSaveLinkDialog(String url, String initialTitle) {
500          showSaveLinkDialog(url, initialTitle, "clipboard", "clipboard", "", true
```

```
501      }
502
503      /**
504       * ■■■■■■■■■■■■■■■■■■■■■■■■■
505       * @param clearClipboardOnSave ■■■■■■■■ true■■■■■■■■■■■■■■■■ false
506       */
507      private void showSaveLinkDialog(String url, String initialTitle,
508              String sourceApp, String originalIntent, String targetActivity,
509              boolean clearClipboardOnSave) {
510          View dialogView = getLayoutInflater().inflate(R.layout.dialog_save_link,
511          EditText titleInput = dialogView.findViewById(R.id.edit_title);
512          EditText urlInput = dialogView.findViewById(R.id.edit_url);
513          EditText tagsInput = dialogView.findViewById(R.id.edit_tags);
514          FlexboxLayout recentTagsContainer = dialogView.findViewById(R.id.recent_
515          Spinner subjectSpinner = dialogView.findViewById(R.id.spinner_save_to_su
516
517          // ■■■■■■■■■
518          titleInput.setHint(initialTitle);  // ■■ hint ■■■■■■■■■
519          urlInput.setText(url);
520
521          // ■■■■■■■■■■■
522          titleInput.requestFocus();
523
524          // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
525          final List<Subject> subjectList = new ArrayList<>();
526          List<String> subjectLabels = new ArrayList<>();
527          subjectLabels.add("■■■■■■■");
528          SubjectDao subjectDao = new SubjectDao(this);
529          subjectDao.open();
530          try {
531              List<Subject> all = subjectDao.getAllSubjects();
532              for (Subject s : all) {
533                  subjectList.add(s);
534                  subjectLabels.add(s.getTitle());
535              }
536          } finally {
537              subjectDao.close();
538          }
539          ArrayAdapter<String> subjectAdapter = new ArrayAdapter<>(this,
540                  android.R.layout.simple_spinner_item, subjectLabels);
541          subjectAdapter.setDropDownViewResource(android.R.layout.simple_spinner_d
542          subjectSpinner.setAdapter(subjectAdapter);
543
544          // ■■■■■■■■■■
545          Context context = this;
546          List<String> recentTags = RecentTagsManager.getRecentTags(context);
547          if (!recentTags.isEmpty()) {
548              TextView recentTagsLabel = dialogView.findViewById(R.id.text_recent_
549              recentTagsLabel.setVisibility(View.VISIBLE);
550
```

```
551             for (String tag : recentTags) {
552                 TextView tagView = (TextView) LayoutInflater.from(context)
553                         .inflate(R.layout.item_recent_tag, recentTagsContainer,
554                 tagView.setText(tag);
555                 tagView.setOnClickListener(v -> {
556                     String currentText = tagsInput.getText().toString().trim();
557                     // ■■■■■■■■■■■■■■■■■■■■■■■■
558                     if (!currentText.isEmpty() && !currentText.endsWith(",") &&
559                         tagsInput.setText(currentText + "■" + tag);
560                     } else {
561                         // ■■■■■■■■■■■■■■■■■■
562                         tagsInput.setText(currentText + tag);
563                     }
564                     // ■■■■■■■■
565                     tagsInput.setSelection(tagsInput.getText().length());
566                 });
567                 recentTagsContainer.addView(tagView);
568             }
569         }
570 
571         AlertDialog dialog = new AlertDialog.Builder(this)
572                 .setTitle("■■■■")
573                 .setView(dialogView)
574                 .setPositiveButton("■■", (d, which) -> {
575                     // ■■■■■■■■■■■■ hint ■■■■■■
576                     String title = titleInput.getText().toString();
577                     if (TextUtils.isEmpty(title)) {
578                         title = initialTitle;
579                     }
580                     String finalUrl = urlInput.getText().toString();
581 
582                     if (!TextUtils.isEmpty(title) && !TextUtils.isEmpty(finalUrl
583                         LinkItem newLink = new LinkItem(
584                             title,
585                             finalUrl,
586                             sourceApp,
587                             originalIntent,
588                             targetActivity
589                         );
590                         String[] tags = tagsInput.getText().toString().split("[,
591                         List<String> tagList = new ArrayList<>();
592                         for (String tag : tags) {
593                             String trimmedTag = tag.trim();
594                             if (!trimmedTag.isEmpty()) {
595                                 tagList.add(trimmedTag);
596                             }
597                         }
598                         newLink.setTags(tagList);
599                         linkDao.insertLink(newLink);
600 
```

```
601                         // ■■■■■■■■■■■■■■■■■■■■■■
602                         int subjectPos = subjectSpinner.getSelectedItemPosition(
603                         if (subjectPos > 0 && subjectPos <= subjectList.size())
604                             Subject chosen = subjectList.get(subjectPos - 1);
605                             SubjectUtil.addLinkToSubjectById(this, chosen.getId(
606                         }
607
608                         // ■■■■■■■■■■■■■■■■■■■■■■■■
609                         if (clearClipboardOnSave) {
610                             ClipboardManager clipboard = (ClipboardManager) getS
611                             clipboard.setPrimaryClip(ClipData.newPlainText("", "
612                         }
613
614                         Snackbar.make(binding.getRoot(), "■■■■" + title,
615                                 Snackbar.LENGTH_LONG).show();
616
617                         NavController navController = Navigation.findNavControll
618                                 R.id.nav_host_fragment_content_main);
619                         navController.navigate(R.id.nav_home);
620
621                         // ■■ HomeFragment ■■■■■■■■■■■■■■■■■■■■
622                         navController.addOnDestinationChangedListener(new NavCon
623                             @Override
624                             public void onDestinationChanged(@NonNull NavControl
625                                                              @NonNull NavDestinat
626                                                              @Nullable Bundle arg
627                                 if (destination.getId() == R.id.nav_home) {
628                                     // ■■ HomeFragment ■■■■■■■■■■
629                                     Fragment fragment = getSupportFragmentManage
630                                             .findFragmentById(R.id.nav_host_frag
631                                     if (fragment instanceof HomeFragment) {
632                                         ((HomeFragment) fragment).recordLinkAddT
633                                     }
634                                     navController.removeOnDestinationChangedList
635                                 }
636                             }
637                         });
638                     }
639                 })
640                 .setNegativeButton("■■", (dialog1, which) -> {
641                     // ■■■■■■■■■■■■■■■■■■■■
642                     // ClipboardManager clipboard = (ClipboardManager) getSystem
643                     // clipboard.setPrimaryClip(ClipData.newPlainText("", ""));
644                 })
645                 .create();
646
647         // ■■■■■■■■■■
648         dialog.setOnDismissListener(dialogInterface -> {
649             // ■■■■■■■■■■■■■■■■■■■■■■■■
650             // ClipboardManager clipboard = (ClipboardManager) getSystemService(
```

```
651             // clipboard.setPrimaryClip(ClipData.newPlainText("", ""));
652         });
653 
654         dialog.show();
655 
656         // ■■■■■■■■■■
657         fetchTitleFromUrl(url, titleInput);
658     }
659 
660     private void fetchTitleFromUrl(String url, EditText titleInput) {
661         ExecutorService executor = Executors.newSingleThreadExecutor();
662         Handler handler = new Handler(Looper.getMainLooper());
663 
664         executor.execute(() -> {
665 
666             String title = "";
667             try {
668                 if (url.contains("weixin.qq.com")) {
669                     title = CrawlUtil.getWeixinArticleTitle(url);
670                 } else {
671                     title = CrawlUtil.fetchTitleCommon(url);
672                 }
673             } catch (Exception e) {
674                 Log.e("FetchTitle", "Error fetching title", e);
675             }
676 
677             final String finalTitle = title;
678             handler.post(() -> {
679                 if (!TextUtils.isEmpty(finalTitle)) {
680                     titleInput.setText(finalTitle);
681                 }
682             });
683         });
684     }
685 
686     @SuppressLint("ResourceType")
687     private void updateNavHeader() {
688         try {
689             NavigationView navigationView = binding.navView;
690             View headerView = navigationView.getHeaderView(0);
691             ImageView profileImage = headerView.findViewById(R.id.nav_header_ima
692             TextView usernameText = headerView.findViewById(R.id.nav_header_user
693             TextView emailText = headerView.findViewById(R.id.nav_header_email);
694 
695             SharedPreferences prefs = getSharedPreferences("UserProfile", MODE_P
696             String username = prefs.getString("username", getString(R.string.nav
697             String email = prefs.getString("email", getString(R.string.nav_heade
698             String imageUri = prefs.getString("profile_image", "");
699 
700             usernameText.setText(username);
```

```
            emailText.setText(email);

            // ■■■■■■■■■■
            if (!TextUtils.isEmpty(imageUri)) {
                try {
                    Uri uri = Uri.parse(imageUri);
                    if (uri.getPath() != null && new File(uri.getPath()).exists(
                        profileImage.setImageURI(null); // ■■■■■■■■
                        profileImage.setImageURI(uri);
                    } else {
                        // ■■■■■■■■■■■■■■■■URI■■■■■■logo
                        prefs.edit().remove("profile_image").apply();
                        profileImage.setImageResource(R.mipmap.ic_launcher);
                    }
                } catch (Exception e) {
                    Log.e("MainActivity", "Error loading profile image", e);
                    profileImage.setImageResource(R.mipmap.ic_launcher);
                }
            }
            // ■■■■■■■■■■■■■■■■■■■■■■ic_launcher■■XML■■■■■■
        } catch (Exception e) {
            Log.e("MainActivity", "Error updating nav header", e);
        }
    }

}
```

```
// MISSING: person/notfresh/readingshare/ShareUtil.java
package person.notfresh.readingshare.ui.home;

import android.app.Activity;
import android.content.Intent;
import android.graphics.Bitmap;
import android.net.Uri;
import android.os.Bundle;
import android.text.Editable;
import android.text.TextWatcher;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.Menu;
import android.view.MenuInflater;
import android.view.MenuItem;
import android.view.View;
import android.view.ViewGroup;
import android.widget.EditText;
import android.widget.PopupWindow;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
```

```
24  import androidx.navigation.Navigation;
25  import androidx.recyclerview.widget.ItemTouchHelper;
26  import androidx.recyclerview.widget.LinearLayoutManager;
27  import androidx.recyclerview.widget.RecyclerView;
28
29  import com.google.android.flexbox.FlexboxLayoutManager;
30  import com.google.android.flexbox.FlexDirection;
31  import com.google.android.flexbox.FlexWrap;
32
33  import com.google.android.material.snackbar.Snackbar;
34
35  import person.notfresh.readingshare.R;
36  import person.notfresh.readingshare.adapter.LinksAdapter;
37  import person.notfresh.readingshare.adapter.SearchHistoryAdapter;
38  import person.notfresh.readingshare.adapter.TagsAdapter;
39  import person.notfresh.readingshare.databinding.FragmentHomeBinding;
40  import person.notfresh.readingshare.db.LinkDao;
41  import person.notfresh.readingshare.db.SearchHistoryManager;
42  import person.notfresh.readingshare.db.SubjectDao;
43  import person.notfresh.readingshare.model.LinkItem;
44  import person.notfresh.readingshare.model.SearchHistoryItem;
45  import person.notfresh.readingshare.core.model.SubjectItem;
46  import person.notfresh.readingshare.core.model.SubjectUtil;
47  import person.notfresh.readingshare.ui.subject.SelectSubjectDialog;
48  import person.notfresh.readingshare.ClickStatisticsActivity;
49  import person.notfresh.readingshare.util.ShareUtil;
50  import person.notfresh.readingshare.util.ImageUtil;
51
52  import android.content.Context;
53  import android.content.SharedPreferences;
54  import android.text.TextUtils;
55  import android.widget.ImageButton;
56  import android.widget.ImageView;
57  import android.widget.ScrollView;
58  import android.widget.TextView;
59  import androidx.appcompat.app.AlertDialog;
60
61  import org.slf4j.Logger;
62  import org.slf4j.LoggerFactory;
63
64  import java.util.ArrayList;
65  import java.util.Collections;
66  import java.util.Date;
67  import java.util.HashSet;
68  import java.util.List;
69  import java.util.Locale;
70  import java.util.Map;
71  import java.util.Set;
72  import java.util.TreeMap;
73  import java.text.SimpleDateFormat;
```

```
 74  import java.util.concurrent.atomic.AtomicInteger;
 75
 76  /**
 77   * ■■Fragment
 78   *
 79   * ■■■■■
 80   * - ■■■■■■■■■■■■■■■■■■■■
 81   * - ■■■■■■■■■■■■■TagsFragment■■■
 82   * - ■■■■■■■■■■■■■■■■■■■■
 83   * - ■■■■■■■
 84   *
 85   * ■■■■■
 86   * - ■■0-2■■TagsFragment■■■■■■■HomeFragment
 87   * - ■■3■■■■■■■
 88   * - ■■4■■■■■■■■■■■■■
 89   *
 90   * @see TagsFragment■■■■■■■■■■■■■Fragment■
 91   */
 92  public class HomeFragment extends Fragment implements LinksAdapter.OnLinkActionL
 93
 94      private static final String TAG = "HomeFragment";  // Logcat■■■■■
 95      private static final int REQUEST_CODE_PICK_ICON = 1001;
 96      private static final Logger log = LoggerFactory.getLogger(HomeFragment.class
 97
 98      private FragmentHomeBinding binding;
 99      private LinksAdapter adapter;
100      private LinkDao linkDao;
101      private boolean isSelectionMode = false;
102      private MenuItem shareMenuItem;
103      private MenuItem closeSelectionMenuItem;
104      private MenuItem enterSelectionMenuItem;
105      private MenuItem selectAllMenuItem;  // ■■■■■■■■TagsFragment■■■
106      private MenuItem addTagMenuItem;  // ■■■■■■■■■■TagsFragment■■■
107      private MenuItem sortMenuItem;  // ■■■■■■■■TagsFragment■■■
108      private MenuItem exitSortMenuItem;  // ■■■■■■■■■■TagsFragment■■■
109      private EditText searchEditText;
110
111      // ========== ■■■■■■■ ==========
112      private static final long SEARCH_DEBOUNCE_MS = 300L;
113      private static final long SEARCH_BLUR_HIDE_DELAY_MS = 200L;
114      private static final int SEARCH_HISTORY_POPUP_MAX_DP = 300;
115      private static final int SEARCH_DEFAULT_END_PADDING_DP = 8;
116      private static final int SEARCH_CLEAR_END_PADDING_DP = 36;
117
118      private SearchHistoryManager searchHistoryManager;
119      private List<SearchHistoryItem> historyCache = new ArrayList<>();
120      private PopupWindow historyPopup;
121      private SearchHistoryAdapter historyAdapter;
122      private final Runnable debounceSearchRunnable = new Runnable() {
123          @Override public void run() { performSearch(searchEditText.getText().toS
```

```
124     };
125     private final Runnable hidePopupRunnable = new Runnable() {
126         @Override public void run() { dismissHistoryPopup(); }
127     };
128     private ImageButton searchClearButton;
129 
130     // ■■■■■■■■■■■■■■■
131     private LinkItem pendingIconItem;
132     private String pendingIconUrl;
133 
134     // ========== ■■■■■■■■■■■■■TagsFragment■■■==========
135     private static final String PREF_NAME = "TagsPreferences";
136     private static final String KEY_SELECTED_TAGS = "selectedTags";
137     private static final String KEY_NO_TAG_SELECTED = "noTagSelected";
138     private static final String NO_TAG = "NO_TAG";  // ■■■■"■■■"■■
139     private static final String PREF_HIGHLIGHTED_TAGS = "highlighted_tags";
140     private static final int FIXED_TAGS_COUNT = 10;  // ■■■■■■■■■■
141     private static final int COLLAPSED_HEIGHT_DP = 120;  // ■■■■
142     private static final int SORT_MODE_MAX_HEIGHT_DP = 400;  // ■■■■■■■■■■■■dp■
143 
144     // ■■■■■■■■■
145     private static final String KEY_SHUFFLE_MODE = "shuffle_mode";
146     private static final long LINK_ADD_INTERVAL_MS = 5000;  // 5■■■■■■■■■■■■■
147 
148     private RecyclerView tagsRecyclerView;
149     private RecyclerView tagsRecyclerViewCollapsed;  // ■■■■■■
150     private TagsAdapter tagsAdapter;
151     private TagsAdapter tagsAdapterCollapsed;  // ■■■■■■■■■■
152     private ItemTouchHelper itemTouchHelper;
153     private View expandMoreTagsButton;  // ■■■■■■■■■
154     private TextView textExpandMore;
155     private ImageView iconExpandMore;
156     private boolean isMoreTagsExpanded = false;  // ■■■■■■■■■
157     private Set<String> selectedTagNames = new HashSet<>();  // ■■Set■■■■■■■■■■
158     private boolean isSortMode = false;  // ■■■■■■■
159     private ScrollView tagsScrollView;
160     private View toggleTagsButton;
161     private ImageView arrowIndicator;
162     private boolean isTagsExpanded = false;  // ■■■■■
163     private Set<String> highlightedTags = new HashSet<>();  // ■■■■■■■■
164     private View tagsContainer;  // ■■■■■■■■■■■■/■■■
165     private boolean isShuffleMode = false;  // ■■■■■■■■■
166     private MenuItem shuffleMenuItem;  // ■■■■■■
167     private MenuItem exitShuffleMenuItem;  // ■■■■■■■■
168     private long lastLinkAddTime = 0;  // ■■■■■■■■■■■■■■■■■■■■■■■■■
169 
170     @Override
171     public void onCreate(@Nullable Bundle savedInstanceState) {
172         super.onCreate(savedInstanceState);
173         Log.d(TAG, "onCreate: start");
```

```
174          try {
175              setHasOptionsMenu(true);
176              Log.d(TAG, "onCreate: setHasOptionsMenu(true) success");
177          } catch (Exception e) {
178              Log.e(TAG, "onCreate: exception", e);
179              throw e;
180          }
181
182          // ■■■■■■■■
183          restoreShuffleModeState();
184      }
185
186      @Override
187      public void onCreateOptionsMenu(@NonNull Menu menu, @NonNull MenuInflater in
188          Log.d(TAG, "onCreateOptionsMenu: start");
189          try {
190              menu.clear();
191              inflater.inflate(R.menu.home_menu, menu);
192              Log.d(TAG, "onCreateOptionsMenu: menu loaded successfully");
193
194              shareMenuItem = menu.findItem(R.id.action_share);
195              closeSelectionMenuItem = menu.findItem(R.id.action_close_selection);
196              enterSelectionMenuItem = menu.findItem(R.id.action_enter_selection);
197              selectAllMenuItem = menu.findItem(R.id.action_select_all);  // ■■■■■■
198              addTagMenuItem = menu.findItem(R.id.action_add_tag);  // ■■■■■■■■
199              sortMenuItem = menu.findItem(R.id.action_sort_tags);  // ■■■■■■
200              exitSortMenuItem = menu.findItem(R.id.action_exit_sort);  // ■■■■■■■■
201              shuffleMenuItem = menu.findItem(R.id.action_shuffle);  // ■■■■■■
202              exitShuffleMenuItem = menu.findItem(R.id.action_exit_shuffle);  // ■
203              MenuItem statisticsMenuItem = menu.findItem(R.id.action_statistics);
204
205              Log.d(TAG, "onCreateOptionsMenu: menu items found - share=" + (share
206                      ", close=" + (closeSelectionMenuItem != null) +
207                      ", enter=" + (enterSelectionMenuItem != null) +
208                      ", selectAll=" + (selectAllMenuItem != null) +
209                      ", addTag=" + (addTagMenuItem != null) +
210                      ", sort=" + (sortMenuItem != null) +
211                      ", exitSort=" + (exitSortMenuItem != null) +
212                      ", shuffle=" + (shuffleMenuItem != null) +
213                      ", exitShuffle=" + (exitShuffleMenuItem != null));
214
215              // ■■■■■■■■■■■■
216              View actionView = requireActivity().findViewById(statisticsMenuItem.
217              if (actionView != null) {
218                  ViewGroup.MarginLayoutParams params = (ViewGroup.MarginLayoutPar
219                  params.rightMargin = getResources().getDimensionPixelSize(R.dime
220                  actionView.setLayoutParams(params);
221              }
222
223             //■■■■■■■■■■■
```

```
224          //     MenuItem clickStatsButton = menu.findItem(R.id.action_statistics_
225          //    clickStatsButton.setOnClickListener(v -> {
226          //        // ■■■■■■■■■
227          //        Intent intent = new Intent(getActivity(), ClickStatisticsActiv
228          //        startActivity(intent);
229          //    });
230
231              // ■■2■■■■■■■■■■■■■■■■■■■■■■
232              updateMenuVisibility();
233
234              super.onCreateOptionsMenu(menu, inflater);
235              Log.d(TAG, "onCreateOptionsMenu: completed");
236          } catch (Exception e) {
237              Log.e(TAG, "onCreateOptionsMenu: exception", e);
238              // ■■■■■■■■■■■■
239              super.onCreateOptionsMenu(menu, inflater);
240          }
241      }
242
243      @Override
244      public boolean onOptionsItemSelected(@NonNull MenuItem item) {
245          int id = item.getItemId();
246          if (id == R.id.action_close_selection) {
247              toggleSelectionMode();  // ■■■■■■■
248              return true;
249          } else if (id == R.id.action_enter_selection) {
250              toggleSelectionMode();  // ■■■■■■■
251              return true;
252          } else if (id == R.id.action_share_text) {
253              shareAsText();
254              return true;
255          } else if (id == R.id.action_share_json) {
256              shareAsFile(true);  // true ■■ JSON
257              return true;
258          } else if (id == R.id.action_share_csv) {
259              shareAsFile(false);  // false ■■ CSV
260              return true;
261          } else if (id == R.id.action_add_to_subject) {
262              addToSubject();
263              return true;
264          } else if (id == R.id.action_statistics) {
265              // ■■■■■■■■
266              Navigation.findNavController(requireView())
267                      .navigate(R.id.action_nav_home_to_nav_statistics);
268              return true;
269          } else if (id == R.id.action_statistics_read) {
270              // ■■■■■■■■■■
271              Intent intent = new Intent(getActivity(), ClickStatisticsActivity.cl
272              startActivity(intent);
273              return true;
```

```
274          } else if (id == R.id.action_add_tag) {
275              // ■■■■■■TagsFragment■■■
276              showAddTagDialog();
277              return true;
278          } else if (id == R.id.action_sort_tags) {
279              // ■■■■■■■■TagsFragment■■■
280              toggleSortMode();
281              return true;
282          } else if (id == R.id.action_exit_sort) {
283              // ■■■■■■■■TagsFragment■■■
284              toggleSortMode();
285              return true;
286          } else if (id == R.id.action_select_all) {
287              // ■■■■TagsFragment■■■
288              selectAllItems();
289              return true;
290          } else if (id == R.id.action_shuffle) {
291              if (isShuffleMode) {
292                  // ■■■■■■■■■■■■
293                  reshuffleLinks();
294              } else {
295                  // ■■■■■■
296                  toggleShuffleMode();
297              }
298              return true;
299          } else if (id == R.id.action_exit_shuffle) {
300              // ■■■■■■
301              toggleShuffleMode();
302              return true;
303          }
304          return super.onOptionsItemSelected(item);
305      }
306
307      @Override
308      public View onCreateView(@NonNull LayoutInflater inflater,
309                               ViewGroup container, Bundle savedInstanceState) {
310          Log.d(TAG, "onCreateView: start");
311          try {
312              binding = FragmentHomeBinding.inflate(inflater, container, false);
313              View root = binding.getRoot();
314              Log.d(TAG, "onCreateView: layout loaded successfully");
315
316              // ■■■■■■■■■■
317              String selectedDate = null;
318              if (getArguments() != null) {
319                  selectedDate = getArguments().getString("selected_date");
320              }
321
322              Log.d(TAG, "onCreateView: initializing LinkDao");
323              linkDao = new LinkDao(requireContext());
```

```
324             linkDao.open();
325             Log.d(TAG, "onCreateView: LinkDao opened successfully");
326 
327             Log.d(TAG, "onCreateView: initializing RecyclerView and Adapter");
328             RecyclerView recyclerView = binding.recyclerView;
329             if (recyclerView == null) {
330                 Log.e(TAG, "onCreateView: recyclerView is null!");
331                 throw new IllegalStateException("recyclerView■null");
332             }
333 
334             adapter = new LinksAdapter(requireContext());
335             adapter.setOnLinkActionListener(this);
336             recyclerView.setAdapter(adapter);
337             recyclerView.setLayoutManager(new LinearLayoutManager(requireContext
338             Log.d(TAG, "onCreateView: RecyclerView setup completed");
339 
340             // ■■■■■■■■■
341             adapter.enableSwipeActions(recyclerView);
342 
343             // ■■ RecyclerView ■■■■■■
344             recyclerView.setOnTouchListener((v, event) -> {
345                 if (searchEditText != null) {
346                     searchEditText.clearFocus();  // ■■■■■■■■■
347                 }
348                 return false;
349             });
350 
351             // ■■■■■■
352             Log.d(TAG, "onCreateView: initializing search box");
353             searchEditText = binding.searchEditText;
354             if (searchEditText != null) {
355                 // ■■■■■■■■
356                 searchHistoryManager = new SearchHistoryManager(requireContext()
357                 refreshHistoryCache();
358 
359                 searchEditText.addTextChangedListener(new TextWatcher() {
360                     @Override public void beforeTextChanged(CharSequence s, int
361                     @Override public void onTextChanged(CharSequence s, int star
362                     @Override
363                     public void afterTextChanged(Editable s) {
364                         // ■■■■■ × ■■■■■■■■■■ 300ms ■■■
365                         updateInputChrome();
366                         // ■■■■■■■■■ + ■■■
367                         searchEditText.removeCallbacks(debounceSearchRunnable);
368                         searchEditText.postDelayed(debounceSearchRunnable, SEARC
369                     }
370                 });
371 
372                 // focus: ■■■ updateInputChrome ■■■■■■■■■■
373                 searchEditText.setOnFocusChangeListener((v, hasFocus) -> {
```

```
374                     if (hasFocus) {
375                         updateInputChrome();
376                     } else {
377                         // 200ms ■■■■■■■■■■■■■
378                         searchEditText.removeCallbacks(hidePopupRunnable);
379                         searchEditText.postDelayed(hidePopupRunnable, SEARCH_BLU
380                     }
381                 });
382 
383                 // × ■■■■■■■■■■ + ■■■■■ + ■ updateInputChrome ■■■■■■■■
384                 searchClearButton = binding.searchClearButton;
385                 if (searchClearButton != null) {
386                     searchClearButton.setOnClickListener(v -> {
387                         searchEditText.removeCallbacks(debounceSearchRunnable);
388                         searchEditText.setText("");
389                         // ■■■■■■setText ■■■■ TextWatcher
390                         updateInputChrome();
391                     });
392                 }
393             } else {
394                 Log.w(TAG, "onCreateView: searchEditText is null");
395             }
396 
397             // ========== ■■■■■■■■UI■■■2■■■==========
398             Log.d(TAG, "onCreateView: initializing tag management UI");
399             try {
400                 initTagRecyclerView(root);
401                 Log.d(TAG, "onCreateView: tag management UI initialized successf
402             } catch (Exception e) {
403                 Log.e(TAG, "onCreateView: tag management UI initialization faile
404                 // ■■■■■■■■■■■■■■
405             }
406 
407             // ========== ■■■■■■■■■■2■==========
408             Log.d(TAG, "onCreateView: start loading data, selectedTagNames.size=
409 
410             // ■■■■■■■■■■■onViewCreated■■■RecyclerView■■■■■■■
411 
412             // ■■■■■■■■■■■■■■■■■■■■■■■
413             // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
414             Log.d(TAG, "onCreateView: start async loading tag list");
415             try {
416                 loadTags();
417             } catch (Exception e) {
418                 Log.e(TAG, "onCreateView: failed to load tag list", e);
419                 // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
420             }
421 
422             // ■■■■■■■■■■■■■■■■
423             if (selectedDate != null) {
```

```
424                 scrollToDate(recyclerView, selectedDate);
425             }
426
427             Log.d(TAG, "onCreateView: completed");
428             return root;
429         } catch (Exception e) {
430             Log.e(TAG, "onCreateView: exception occurred", e);
431             throw e;
432         }
433     }
434
435     /**
436      * ■■■■■■■ + ■■■■ + ■■■■■
437      * ■ debounce ■■■■■■■■■■■■■
438      */
439     private void performSearch(String rawText) {
440         if (adapter == null) return;
441         String text = rawText == null ? "" : rawText.trim();
442         adapter.filter(text);
443         // ■■■■■■■■■■
444         if (!text.isEmpty() && searchHistoryManager != null) {
445             searchHistoryManager.addSearchKeyword(text);
446             refreshHistoryCache();
447             updateInputChrome();
448         }
449     }
450
451     /** ■■■■■■■■■■■■■■■■■■■■■■ */
452     private void refreshHistoryCache() {
453         if (searchHistoryManager == null) return;
454         historyCache = searchHistoryManager.loadHistory();
455         if (historyAdapter != null) {
456             historyAdapter.setItems(historyCache);
457         }
458     }
459
460     /** dp → px ■■■■■■ setPadding■ */
461     private int dpToPx(int dp) {
462         return (int) (dp * getResources().getDisplayMetrics().density);
463     }
464
465     /**
466      * ■■■■■■■■■■■ × ■■■■■■■■■
467      * - ■■■■■■■ ×■■■■■■■
468      * - ■ + ■■ + ■■■■■■■■■
469      * - ■ + ■■■■■■■■■■■
470      */
471     private void updateInputChrome() {
472         if (searchEditText == null) return;
473         String text = searchEditText.getText().toString();
```

```
474          boolean hasText = !text.isEmpty();
475
476          if (searchClearButton != null) {
477              searchClearButton.setVisibility(hasText ? View.VISIBLE : View.GONE);
478          }
479          // ■■ paddingEnd■× ■■ 36dp ■■■× ■■■ 8dp ■■■■
480          // paddingStart / Top / Bottom ■■ 8dp ■■■ XML
481          int endPaddingPx = dpToPx(hasText ? SEARCH_CLEAR_END_PADDING_DP : SEARCH
482          searchEditText.setPaddingRelative(
483              searchEditText.getPaddingStart(),
484              searchEditText.getPaddingTop(),
485              endPaddingPx,
486              searchEditText.getPaddingBottom()
487          );
488
489          if (!hasText && searchEditText.hasFocus() && !historyCache.isEmpty()) {
490              showHistoryPopup();
491          } else {
492              dismissHistoryPopup();
493          }
494      }
495
496      /** ■■■■■■■ EditText ■■■■■■■■■■■■ */
497      private void showHistoryPopup() {
498          if (searchEditText == null || searchEditText.getWindowToken() == null) r
499          if (historyCache.isEmpty()) return;   // ■■■■■■■■
500          refreshHistoryCache();               // ■■■■■■■■■■■■■■■■■■■■
501          if (historyPopup == null) {
502              buildHistoryPopup();
503          }
504          if (historyPopup != null && !historyPopup.isShowing()) {
505              historyPopup.showAsDropDown(searchEditText, 0, 0);
506          }
507      }
508
509      private void dismissHistoryPopup() {
510          if (historyPopup != null && historyPopup.isShowing()) {
511              historyPopup.dismiss();
512          }
513      }
514
515      /** ■■ PopupWindow■■■■■■■ */
516      private void buildHistoryPopup() {
517          Context ctx = requireContext();
518          historyAdapter = new SearchHistoryAdapter();
519          historyAdapter.setOnItemClickListener(new SearchHistoryAdapter.OnItemCli
520              @Override public void onItemClick(SearchHistoryItem item) {
521                  String kw = item.getText();
522                  searchEditText.removeCallbacks(hidePopupRunnable);
523                  searchEditText.setText(kw);
```

```
524                 searchEditText.setSelection(kw.length());
525                 searchEditText.removeCallbacks(debounceSearchRunnable);
526                 dismissHistoryPopup();
527                 // ■■■■■■■■setText ■■■ TextWatcher
528                 performSearch(kw);
529                 updateInputChrome();
530             }
531             @Override public void onPinClick(SearchHistoryItem item) {
532                 searchHistoryManager.togglePinKeyword(item.getText());
533                 refreshHistoryCache();
534                 updateInputChrome();
535             }
536             @Override public void onDeleteClick(SearchHistoryItem item) {
537                 searchHistoryManager.deleteKeyword(item.getText());
538                 refreshHistoryCache();
539                 updateInputChrome();
540             }
541         });
542         historyAdapter.setItems(historyCache);
543 
544         RecyclerView rv = new RecyclerView(ctx);
545         rv.setLayoutManager(new LinearLayoutManager(ctx));
546         rv.setAdapter(historyAdapter);
547         rv.setBackgroundResource(R.drawable.search_history_popup_bg);
548         rv.setElevation(8f);
549 
550         int width = searchEditText.getWidth();
551         int maxHeight = (int) (SEARCH_HISTORY_POPUP_MAX_DP * ctx.getResources().
552         historyPopup = new PopupWindow(rv, width, ViewGroup.LayoutParams.WRAP_CO
553         historyPopup.setBackgroundDrawable(new android.graphics.drawable.ColorDr
554         historyPopup.setOutsideTouchable(false);
555         historyPopup.setFocusable(false);  // ■■■■■■■■■■ EditText ■■
556         historyPopup.setHeight(maxHeight);
557 
558         // ■■■■■■■■■■■■ pending hide■spec 6.6 ■■■■■
559         rv.setOnTouchListener((v, ev) -> {
560             searchEditText.removeCallbacks(hidePopupRunnable);
561             return false;
562         });
563     }
564 
565     @Override
566     public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstance
567         super.onViewCreated(view, savedInstanceState);
568         Log.d(TAG, "onViewCreated: start");
569 
570         // ■onViewCreated■■■■■■■■■■■■■RecyclerView■■■■■■■■■
571         // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
572         if (linkDao != null && adapter != null) {
573             Log.d(TAG, "onViewCreated: loading all links directly");
```

```
574             List<LinkItem> pinnedLinks = linkDao.getPinnedLinks();
575             Map<String, List<LinkItem>> groupedLinks = linkDao.getLinksGroupByDa
576
577             // ■■■■■■
578             int totalLinks = pinnedLinks.size();
579             for (List<LinkItem> links : groupedLinks.values()) {
580                 totalLinks += links.size();
581             }
582
583             Log.d(TAG, "onViewCreated: loaded links - pinned=" + pinnedLinks.siz
584                     + ", groups=" + groupedLinks.size() + ", total links=" + tot
585
586             // ■■■■■■■adapter
587             adapter.setPinnedLinks(pinnedLinks);
588             adapter.setGroupedLinks(groupedLinks);
589             Log.d(TAG, "onViewCreated: data set to adapter, itemCount=" + adapte
590
591             // ■■RecyclerView■■
592             RecyclerView recyclerView = binding.recyclerView;
593             if (recyclerView != null) {
594                 Log.d(TAG, "onViewCreated: RecyclerView state - visibility=" + r
595                         + " (0=VISIBLE), width=" + recyclerView.getWidth()
596                         + ", height=" + recyclerView.getHeight()
597                         + ", isShown=" + recyclerView.isShown()
598                         + ", hasFixedSize=" + recyclerView.hasFixedSize());
599
600                 // ■■RecyclerView■■
601                 if (recyclerView.getVisibility() != View.VISIBLE) {
602                     Log.w(TAG, "onViewCreated: RecyclerView is not visible, sett
603                     recyclerView.setVisibility(View.VISIBLE);
604                 }
605
606                 // ■■LayoutManager
607                 RecyclerView.LayoutManager layoutManager = recyclerView.getLayou
608                 if (layoutManager != null) {
609                     Log.d(TAG, "onViewCreated: LayoutManager exists, itemCount="
610                 } else {
611                     Log.e(TAG, "onViewCreated: LayoutManager is null!");
612                 }
613
614                 // ■■■■
615                 recyclerView.post(() -> {
616                     Log.d(TAG, "onViewCreated: post - RecyclerView width=" + rec
617                             + ", height=" + recyclerView.getHeight()
618                             + ", adapter itemCount=" + (adapter != null ? adapte
619                     if (adapter != null) {
620                         adapter.notifyDataSetChanged();
621                         Log.d(TAG, "onViewCreated: post - notifyDataSetChanged c
622                     }
623                 });
```

```
624             } else {
625                 Log.e(TAG, "onViewCreated: RecyclerView is null!");
626             }
627         } else {
628             Log.e(TAG, "onViewCreated: linkDao or adapter is null!");
629         }
630 
631         // ■■■■■■■■■■■■■■■■■■■■
632         if (getArguments() != null) {
633             String selectedDate = getArguments().getString("selected_date");
634             if (selectedDate != null && binding != null) {
635                 RecyclerView recyclerView = binding.recyclerView;
636                 if (recyclerView != null) {
637                     scrollToDate(recyclerView, selectedDate);
638                 }
639             }
640         }
641 
642         Log.d(TAG, "onViewCreated: completed");
643     }
644 
645     // ========== ■■■■■■■■■■■ ==========
646 
647     /**
648      * ■■■■■RecyclerView
649      * ■TagsFragment■■■■■2■■■■■■■■
650      */
651     private void initTagRecyclerView(View root) {
652         Log.d(TAG, "initTagRecyclerView: start");
653         try {
654             if (root == null) {
655                 Log.e(TAG, "initTagRecyclerView: root is null!");
656                 return;
657             }
658 
659             // ■■■■■■■■■■2■■■■■
660             tagsContainer = root.findViewById(R.id.tags_container);
661             if (tagsContainer != null) {
662                 tagsContainer.setVisibility(View.VISIBLE);  // ■■2■■■■■■■■■■
663                 Log.d(TAG, "initTagRecyclerView: tags container displayed");
664             } else {
665                 Log.w(TAG, "initTagRecyclerView: tagsContainer is null, may not
666             }
667 
668             // ■■■■■■■■ RecyclerView
669             Log.d(TAG, "initTagRecyclerView: initializing fixed tags RecyclerVie
670             tagsRecyclerView = root.findViewById(R.id.recycler_tags_fixed);
671             if (tagsRecyclerView != null) {
672                 try {
673                     FlexboxLayoutManager layoutManager = new FlexboxLayoutManage
```

```
674                     layoutManager.setFlexDirection(FlexDirection.ROW);
675                     layoutManager.setFlexWrap(FlexWrap.WRAP);
676                     tagsRecyclerView.setLayoutManager(layoutManager);
677
678                     tagsAdapter = new TagsAdapter();
679                     tagsAdapter.setOnTagClickListener((position, tag) -> {
680                         if (!isSortMode) {
681                             handleTagClick(tag.getName());
682                         }
683                     });
684                     tagsAdapter.setOnTagLongClickListener((position, tag) -> {
685                         if (!isSortMode) {
686                             showTagOptionsDialog(tag.getName());
687                         }
688                     });
689                     tagsRecyclerView.setAdapter(tagsAdapter);
690                     Log.d(TAG, "initTagRecyclerView: fixed tags RecyclerView ini
691                 } catch (Exception e) {
692                     Log.e(TAG, "initTagRecyclerView: fixed tags RecyclerView ini
693                 }
694             } else {
695                 Log.w(TAG, "initTagRecyclerView: recycler_tags_fixed is null");
696             }
697
698             // ■■■■■■■ RecyclerView
699             Log.d(TAG, "initTagRecyclerView: initializing collapsed tags Recycle
700             tagsRecyclerViewCollapsed = root.findViewById(R.id.recycler_tags_col
701             if (tagsRecyclerViewCollapsed != null) {
702                 try {
703                     FlexboxLayoutManager layoutManagerCollapsed = new FlexboxLay
704                     layoutManagerCollapsed.setFlexDirection(FlexDirection.ROW);
705                     layoutManagerCollapsed.setFlexWrap(FlexWrap.WRAP);
706                     tagsRecyclerViewCollapsed.setLayoutManager(layoutManagerColl
707
708                     tagsAdapterCollapsed = new TagsAdapter();
709                     tagsAdapterCollapsed.setOnTagClickListener((position, tag) -
710                         if (!isSortMode) {
711                             handleTagClick(tag.getName());
712                         }
713                     });
714                     tagsAdapterCollapsed.setOnTagLongClickListener((position, ta
715                         if (!isSortMode) {
716                             showTagOptionsDialog(tag.getName());
717                         }
718                     });
719                     tagsRecyclerViewCollapsed.setAdapter(tagsAdapterCollapsed);
720                     Log.d(TAG, "initTagRecyclerView: collapsed tags RecyclerView
721                 } catch (Exception e) {
722                     Log.e(TAG, "initTagRecyclerView: collapsed tags RecyclerView
723                 }
```

```
724             } else {
725                 Log.w(TAG, "initTagRecyclerView: recycler_tags_collapsed is null
726             }
727 
728         // ■■■■■■■■■■■■
729         expandMoreTagsButton = root.findViewById(R.id.btn_expand_more_tags);
730         if (expandMoreTagsButton != null) {
731             textExpandMore = root.findViewById(R.id.text_expand_more);
732             iconExpandMore = root.findViewById(R.id.icon_expand_more);
733             expandMoreTagsButton.setOnClickListener(v -> toggleMoreTagsExpansion
734         }
735 
736         // ■■■■ ItemTouchHelper■■■■■■■■■
737         if (tagsRecyclerView != null) {
738             ItemTouchHelper.Callback callback = new ItemTouchHelper.SimpleCallba
739                 ItemTouchHelper.UP | ItemTouchHelper.DOWN |
740                 ItemTouchHelper.LEFT | ItemTouchHelper.RIGHT,
741                 0
742             ) {
743                 @Override
744                 public boolean onMove(@NonNull RecyclerView recyclerView,
745                                       @NonNull RecyclerView.ViewHolder viewHolder
746                                       @NonNull RecyclerView.ViewHolder target) {
747                     if (isSortMode && recyclerView == tagsRecyclerView) {
748                         int fromPos = viewHolder.getAdapterPosition();
749                         int toPos = target.getAdapterPosition();
750                         tagsAdapter.swapItems(fromPos, toPos);
751                         saveTagOrderToDatabase();
752                         return true;
753                     }
754                     return false;
755                 }
756 
757                 @Override
758                 public void onSwiped(@NonNull RecyclerView.ViewHolder viewHolder
759                     // ■■■■■
760                 }
761 
762                 @Override
763                 public boolean isLongPressDragEnabled() {
764                     return isSortMode;  // ■■■■■■■■■■
765                 }
766             };
767             itemTouchHelper = new ItemTouchHelper(callback);
768             itemTouchHelper.attachToRecyclerView(tagsRecyclerView);
769         }
770 
771         // ■■■■■■■■■■/■■■■■■■
772         tagsScrollView = root.findViewById(R.id.tags_scrollview);
773         toggleTagsButton = root.findViewById(R.id.btn_toggle_tags);
```

```
774          arrowIndicator = root.findViewById(R.id.arrow_indicator);
775
776          if (tagsScrollView != null) {
777              ViewGroup.LayoutParams params = tagsScrollView.getLayoutParams();
778              params.height = ViewGroup.LayoutParams.WRAP_CONTENT;
779              tagsScrollView.setLayoutParams(params);
780          }
781
782          if (toggleTagsButton != null) {
783              toggleTagsButton.setOnClickListener(v -> toggleTagsExpansion());
784          }
785
786          if (arrowIndicator != null) {
787              arrowIndicator.setImageResource(R.drawable.ic_expand_less);
788              arrowIndicator.setContentDescription("■■■■");
789          }
790
791              // ■■■■■■■■
792              try {
793                  loadHighlightedTags();
794              } catch (Exception e) {
795                  Log.e(TAG, "initTagRecyclerView: failed to load highlighted tags
796              }
797
798              Log.d(TAG, "initTagRecyclerView: completed");
799          } catch (Exception e) {
800              Log.e(TAG, "initTagRecyclerView: exception occurred", e);
801              throw e;
802          }
803      }
804
805      @Override
806      public void onDestroyView() {
807          Log.d(TAG, "onDestroyView: start");
808          super.onDestroyView();
809          if (searchEditText != null) {
810              searchEditText.removeCallbacks(debounceSearchRunnable);
811              searchEditText.removeCallbacks(hidePopupRunnable);
812          }
813          dismissHistoryPopup();
814          historyPopup = null;
815          historyAdapter = null;
816          searchClearButton = null;   // ■■
817          binding = null;
818          if (linkDao != null) {
819              linkDao.close();
820              Log.d(TAG, "onDestroyView: LinkDao closed");
821          }
822          Log.d(TAG, "onDestroyView: completed");
823      }
```

```
824
825     // ========== ■■■■■■■■■■■■ ==========
826
827     @Override
828     public boolean deleteLink(Long linkId){
829         Log.d("HomeFragment", "deleteLink: + link id " + linkId);
830         linkDao.deleteLink(linkId);
831         refreshLinksList();
832         return true;
833     }
834
835     @Override
836     public void onDeleteLink(LinkItem link) {
837         Log.d("HomeFragment", "onDeleteLink: " + link.getTitle() + ", link id "
838
839         // ■■■■■■■■■■
840         linkDao.deleteLink(link.getId());
841
842         // ■■■■■■■■■■■■■■■■■■■■■■
843         boolean removed = adapter.removeLinkItem(link);
844         Log.d("HomeFragment", "onDeleteLink: removeLinkItem returned " + removed
845
846         // ■■■■■■■■■■/■■■■■■■■■■■■■■
847         if (isShuffleMode) {
848             isShuffleMode = false;
849             saveShuffleModeState();
850             updateMenuVisibility();
851         }
852
853         // ■■■■■■■■■■■■■■■■■■■■■■■■■
854         // ■■■■■■■■■■■■■■■■■■■■■■
855         // if (tagsContainer != null && tagsContainer.getVisibility() == View.VI
856         //     loadTags();
857         // }
858
859         Log.d("HomeFragment", "onDeleteLink completed, adapter itemCount=" + ada
860     }
861
862     @Override
863     public void onUpdateLink(LinkItem oldLink, String newTitle) {
864         linkDao.updateLinkTitle(oldLink.getUrl(), newTitle);
865         // ■■■■■■■■■■/■■■■■■■■■■■■■■
866         if (isShuffleMode) {
867             isShuffleMode = false;
868             saveShuffleModeState();
869             updateMenuVisibility();
870         }
871         // ■■■■■■■■■■■■■■■■■■■■■■
872         refreshLinksList();
873     }
```

```
874
875      //@Override
876      public void addTagToLink(LinkItem item, String tag) {
877          linkDao.addTagToLink(item.getId(), tag);
878          // ■■■■■■■■■■■■■■■■■■■■■■■
879          refreshLinksList();
880          // ■■■■■■■■■■■■■■■■■■■■■■■■■
881          if (tagsContainer != null && tagsContainer.getVisibility() == View.VISIB
882              loadTags();
883          }
884      }
885
886      @Override
887      public void addTagsToLink(LinkItem item, List<String> tags) {
888          // TODO: ■■■■■■■■■■
889      }
890
891      //@Override
892      public void updateLinkTags(LinkItem item) {
893          linkDao.updateLinkTags(item);
894          // ■■■■■■■■■■■■■■■■■■■■■■■
895          refreshLinksList();
896          // ■■■■■■■■■■■■■■■■■■■■■■■■■
897          if (tagsContainer != null && tagsContainer.getVisibility() == View.VISIB
898              loadTags();
899          }
900      }
901
902      @Override
903      public void onEnterSelectionMode() {
904          if (!isSelectionMode) {
905              toggleSelectionMode();
906          }
907      }
908
909      // ========== ■■■■■■■■■ ==========
910
911      /**
912       * ■■■■■■■
913       * ■■■■■■■■■■■■■■■■■■■■■■■■■■
914       * ■■2■■■■■■■■■■■■■■■■■■■■
915       */
916      private void toggleSelectionMode() {
917          Log.d("HomeFragment", "toggleSelectionMode called");
918          isSelectionMode = !isSelectionMode;
919          adapter.toggleSelectionMode();
920
921          // ■■■■■■■■■■■
922          updateMenuVisibility();
923
```

```
        // [redacted]
        if (isSelectionMode) {
            requireActivity().setTitle("[redacted]");
        } else {
            // [redacted]
            if (selectedTagNames.isEmpty()) {
                requireActivity().setTitle(R.string.app_name);
            } else {
                // [redacted]updateContentBySelectedTags[redacted]
            }
        }
        requireActivity().invalidateOptionsMenu();
        Log.d("HomeFragment", "Selection mode: " + isSelectionMode);
    }

    /**
     * [redacted]
     * [redacted]2[redacted]
     */
    private void updateMenuVisibility() {
        // [redacted]
        if (shareMenuItem != null) {
            shareMenuItem.setVisible(isSelectionMode);
        }
        if (closeSelectionMenuItem != null) {
            closeSelectionMenuItem.setVisible(isSelectionMode);
        }
        if (enterSelectionMenuItem != null) {
            enterSelectionMenuItem.setVisible(!isSelectionMode && !isSortMode);
        }
        if (selectAllMenuItem != null) {
            selectAllMenuItem.setVisible(isSelectionMode);
        }

        // [redacted]
        boolean tagsVisible = tagsContainer != null && tagsContainer.getVisibili
        if (addTagMenuItem != null) {
            addTagMenuItem.setVisible(tagsVisible && !isSelectionMode && !isSort
        }
        if (sortMenuItem != null) {
            sortMenuItem.setVisible(tagsVisible && !isSelectionMode && !isSortMo
        }
        if (exitSortMenuItem != null) {
            exitSortMenuItem.setVisible(isSortMode);
        }

        // [redacted]
        // [redacted]
        // [redacted]
        Log.d(TAG, "updateMenuVisibility: isSelectionMode=" + isSelectionMode +
```

```
974          if (shuffleMenuItem != null) {
975              shuffleMenuItem.setVisible(!isSelectionMode && !isSortMode);
976              Log.d(TAG, "updateMenuVisibility: shuffleMenuItem visible=" + shuffl
977          }
978          if (exitShuffleMenuItem != null) {
979              exitShuffleMenuItem.setVisible(isShuffleMode);
980              Log.d(TAG, "updateMenuVisibility: exitShuffleMenuItem visible=" + ex
981          }
982      }
983
984      /**
985       * ■■■■■■■
986       * ■TagsFragment■■■■■■■■■■
987       * ■■2■■■■■■■■■■■■■■■■■■■■
988       */
989      private void toggleSortMode() {
990          isSortMode = !isSortMode;
991
992          // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
993          if (isSortMode && isSelectionMode) {
994              isSelectionMode = false;
995              adapter.toggleSelectionMode();
996          }
997
998          // ■■■■■■■■■■■
999          updateMenuVisibility();
1000
1001         // ■■/■■■■■■■
1002         if (itemTouchHelper != null) {
1003             if (isSortMode) {
1004                 // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1005                 itemTouchHelper.attachToRecyclerView(tagsRecyclerView);
1006             } else {
1007                 itemTouchHelper.attachToRecyclerView(null);
1008             }
1009         }
1010
1011         // ■■/■■■■■■■■■■■■■■■■■■
1012         if (toggleTagsButton != null) {
1013             toggleTagsButton.setVisibility(isSortMode ? View.GONE : View.VISIBLE
1014         }
1015
1016         // ■■■■■■■■■
1017         updateTagsScrollViewHeight();
1018
1019         // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1020         loadTags();
1021
1022         // ■■■■■
1023         if (isSortMode) {
```

```
1024              requireActivity().setTitle("■■■■■■■");
1025              Toast.makeText(requireContext(), "■■■■■■■■■■", Toast.LENGTH_SHORT).sh
1026          } else {
1027              // ■■■■■■■■■■■■■■■
1028              if (selectedTagNames.isEmpty()) {
1029                  requireActivity().setTitle(R.string.app_name);
1030              } else {
1031                  // ■■■■■updateContentBySelectedTags■■■
1032              }
1033          }
1034
1035          requireActivity().invalidateOptionsMenu();
1036      }
1037
1038      /**
1039       * ■■■■■■■■■■
1040       * ■■■■ refreshLinksList ■■■■■■■■■■■
1041       */
1042      public void recordLinkAddTime() {
1043          lastLinkAddTime = System.currentTimeMillis();
1044      }
1045
1046      // ========== ■■■■■ ==========
1047
1048      /**
1049       * ■■■■■■■
1050       * ■■■■■■■■■■■■■■■■■■■■■■
1051       * ■■■■■■■■■■■■■■■■
1052       */
1053      private void toggleShuffleMode() {
1054          Log.d(TAG, "toggleShuffleMode: start, current isShuffleMode=" + isShuffl
1055          isShuffleMode = !isShuffleMode;
1056          Log.d(TAG, "toggleShuffleMode: toggled to isShuffleMode=" + isShuffleMod
1057
1058          // ■■■■■■■■■
1059          saveShuffleModeState();
1060
1061          // ■■■■■■■■■■■■■■■
1062          refreshLinksList();
1063
1064          // ■■■■■■■■
1065          updateMenuVisibility();
1066
1067          // ■■■■■
1068          if (isShuffleMode) {
1069              requireActivity().setTitle("■■■■■");
1070              Toast.makeText(requireContext(), "■■■■■■■■", Toast.LENGTH_SHORT).show
1071          } else {
1072              requireActivity().setTitle(R.string.app_name);
1073          }
```

```
1074
1075          requireActivity().invalidateOptionsMenu();
1076          Log.d(TAG, "toggleShuffleMode: completed, isShuffleMode=" + isShuffleMod
1077      }
1078
1079      /**
1080       * ■■■■■■■■■■■■■■■■■■■
1081       * ■■■■■■■■■■■■■■■
1082       */
1083      private void reshuffleLinks() {
1084          if (!isShuffleMode) {
1085              return;
1086          }
1087          Log.d(TAG, "reshuffleLinks: reshuffling links");
1088
1089          // ■■■■■■■■■■■■■
1090          refreshLinksList();
1091
1092          Toast.makeText(requireContext(), "■■■■■", Toast.LENGTH_SHORT).show();
1093      }
1094
1095      /**
1096       * ■■■■■■■■■■SharedPreferences
1097       */
1098      private void saveShuffleModeState() {
1099          requireContext().getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
1100                  .edit()
1101                  .putBoolean(KEY_SHUFFLE_MODE, isShuffleMode)
1102                  .apply();
1103      }
1104
1105      /**
1106       * ■■■■■■■■■■SharedPreferences
1107       */
1108      private void restoreShuffleModeState() {
1109          isShuffleMode = requireContext().getSharedPreferences(PREF_NAME, Context
1110                  .getBoolean(KEY_SHUFFLE_MODE, false);
1111          Log.d(TAG, "restoreShuffleModeState: isShuffleMode=" + isShuffleMode);
1112      }
1113
1114      // ========== ■■■■■■■■■■■ ==========
1115
1116      /**
1117       * ■■■■■
1118       * ■■■■■■■■■■■■■
1119       */
1120      private void shareAsText() {
1121          Set<LinkItem> selectedItems = adapter.getSelectedItems();
1122          ShareUtil.shareLinksAsText(requireContext(), new ArrayList<>(selectedIte
1123      }
```

```
1124
1125      /**
1126       * ■■■■
1127       * ■■■■■■■■■■■■■■JSON■CSV■
1128       * @param isJson true■■JSON■■■false■■CSV■■
1129       */
1130      private void shareAsFile(boolean isJson) {
1131          Set<LinkItem> selectedItems = adapter.getSelectedItems();
1132          ArrayList<LinkItem> items = new ArrayList<>(selectedItems);
1133          ShareUtil.shareLinksAsFileWithDialog(requireContext(), items, isJson, de
1134              if (deleteAfterShare) {
1135                  // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1136                  for (LinkItem item : items) {
1137                      linkDao.deleteLink(item.getId());
1138                      adapter.removeLinkItem(item);
1139                  }
1140                  // ■■■■■
1141                  refreshLinksList();
1142                  Toast.makeText(requireContext(), "■■■■■■■■■■", Toast.LENGTH_SHORT
1143              }
1144          });
1145      }
1146
1147      /**
1148       * ■■■■■■
1149       * ■■■■■■■■■■■■
1150       */
1151      private void addToSubject() {
1152          Set<LinkItem> selectedItems = adapter.getSelectedItems();
1153          if (selectedItems.isEmpty()) {
1154              Toast.makeText(requireContext(), "■■■■■■■■■■■", Toast.LENGTH_SHORT).s
1155              return;
1156          }
1157          // ■■■■■■■
1158          addLinksToSubject(new ArrayList<>(selectedItems));
1159      }
1160
1161      /**
1162       * ■■
1163       * ■■■■■■■■■■■■■■■■■■■■■■■■
1164       * ■■2■■■■■■■■■■■■■■■■■■
1165       */
1166      private void selectAllItems() {
1167          List<LinkItem> links;
1168
1169          if (selectedTagNames.isEmpty()) {
1170              // ■■■■■■■■■■■■■■■■■
1171              links = linkDao.getAllLinks();
1172          } else {
1173              // ■■■■■■■■■■
```

```
1174              Set<String> tagNames = new HashSet<>();
1175              boolean hasNoTagFilter = false;
1176
1177              for (String tagName : selectedTagNames) {
1178                  if (NO_TAG.equals(tagName)) {
1179                      hasNoTagFilter = true;
1180                  } else {
1181                      tagNames.add(tagName);
1182                  }
1183              }
1184
1185              // ■■■■■■■■■■■■
1186              links = new ArrayList<>();
1187              if (hasNoTagFilter) {
1188                  links.addAll(linkDao.getLinksWithoutTags());
1189              }
1190              if (!tagNames.isEmpty()) {
1191                  links.addAll(linkDao.getLinksByTags(tagNames));
1192              }
1193          }
1194
1195          // ■■■■■■■■■■■■
1196          for (LinkItem item : links) {
1197              adapter.selectItem(item);
1198          }
1199          adapter.notifyDataSetChanged();
1200      }
1201
1202      /**
1203       * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1204       * ■■■■■■■■■■■■
1205       */
1206      private void addLinksToSubject(List<LinkItem> items) {
1207          if (items == null || items.isEmpty()) {
1208              Toast.makeText(requireContext(), "■■■■■■■■■■■■", Toast.LENGTH_SHORT).s
1209              return;
1210          }
1211
1212          // ■■■■ID■■
1213          List<Long> linkIds = new ArrayList<>();
1214          for (LinkItem item : items) {
1215              linkIds.add(item.getId());
1216          }
1217
1218          // ■■■■■■■■■■
1219          SelectSubjectDialog dialog = SelectSubjectDialog.newInstance(linkIds);
1220          dialog.setOnSubjectSelectedListener((subjectId, selectedLinkIds) -> {
1221              // ■■■■ SubjectItem
1222              SubjectDao subjectDao = new SubjectDao(requireContext());
1223              subjectDao.open();
```

```
1224              try {
1225                  // ■■■■■■■■■■■■■ orderIndex
1226                  List<SubjectItem> existingItems = subjectDao.getSubjectItemsBySu
1227
1228                  // ■■■■■■■■ SubjectItem
1229                  List<SubjectItem> newItems = new ArrayList<>();
1230                  for (Long linkId : selectedLinkIds) {
1231                      SubjectItem item = new SubjectItem(subjectId);
1232                      item.setLinkId(linkId);
1233                      // ■■ orderIndex
1234                      int orderIndex = SubjectUtil.calculateOrderIndex(existingIte
1235                      item.setOrderIndex(orderIndex);
1236                      existingItems.add(item); // ■■■■■■■■■■■■■■■■
1237                      newItems.add(item);
1238                  }
1239
1240                  // ■■■■■
1241                  subjectDao.batchInsertSubjectItems(newItems);
1242                  Toast.makeText(requireContext(), "■■■ " + newItems.size() + " ■■
1243              } finally {
1244                  subjectDao.close();
1245              }
1246          });
1247          dialog.show(getParentFragmentManager(), "SelectSubjectDialog");
1248      }
1249
1250      @Override
1251      public void onAddToSubject(LinkItem item) {
1252          // ■■■■■■■■■■■■■■■■■■
1253          List<LinkItem> items = new ArrayList<>();
1254          items.add(item);
1255          addLinksToSubject(items);
1256      }
1257
1258      @Override
1259      public void onPinStatusChanged() {
1260          Log.d("HomeFragment", "onPinStatusChanged called");
1261          // ■■■■■■■■■■■■■■■■■■■■■■■■
1262          refreshLinksList();
1263      }
1264
1265      // ========== ■■■■■■■■■■ ==========
1266
1267      /**
1268       * ■■■■■■■
1269       * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1270       * - ■■■■■■■■■■■■■■■■■■■■■■■■■
1271       * - ■■■■■■■■■■■■■■■■■■■■■■■■■■
1272       */
1273      private void refreshLinksList() {
```

```
1274          Log.d(TAG, "refreshLinksList: start, selectedTagNames.size=" + selectedT
1275          try {
1276              if (linkDao == null) {
1277                  Log.e(TAG, "refreshLinksList: linkDao is null!");
1278                  return;
1279              }
1280              if (adapter == null) {
1281                  Log.e(TAG, "refreshLinksList: adapter is null!");
1282                  return;
1283              }
1284
1285              if (selectedTagNames.isEmpty()) {
1286                  // ■■■■■■■■■■■■HomeFragment■■■■■■
1287                  List<LinkItem> pinnedLinks = linkDao.getPinnedLinks();
1288                  List<LinkItem> allLinks = linkDao.getAllLinks();
1289
1290                  // ■■■■■■■■
1291                  Set<Long> pinnedIds = new HashSet<>();
1292                  for (LinkItem p : pinnedLinks) {
1293                      pinnedIds.add(p.getId());
1294                  }
1295                  List<LinkItem> normalLinks = new ArrayList<>();
1296                  for (LinkItem link : allLinks) {
1297                      if (!pinnedIds.contains(link.getId())) {
1298                          normalLinks.add(link);
1299                      }
1300                  }
1301
1302                  // ■■■■■■■■■■■5■■■■
1303                  boolean recentlyAdded = (System.currentTimeMillis() - lastLinkAd
1304
1305                  // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1306                  if (isShuffleMode && !recentlyAdded) {
1307                      Log.d(TAG, "refreshLinksList: shuffling " + normalLinks.size
1308                      Collections.shuffle(normalLinks);
1309                  } else {
1310                      Log.d(TAG, "refreshLinksList: NOT shuffling, isShuffleMode="
1311                  }
1312
1313                  // ■■■■■■■■■■■■
1314                  Map<String, List<LinkItem>> groupedLinks;
1315                  if (isShuffleMode && !recentlyAdded) {
1316                      // ■■■■■■■■■■■■■■■■■■■■
1317                      groupedLinks = null;
1318                  } else {
1319                      // ■■■■■■■■■■■■■■■■■■■■
1320                      groupedLinks = groupLinksByDate(normalLinks);
1321                  }
1322
1323                  // ■■■■■■■
```

```
1324                int totalLinks = pinnedLinks.size();
1325                if (groupedLinks != null) {
1326                    for (List<LinkItem> links : groupedLinks.values()) {
1327                        totalLinks += links.size();
1328                    }
1329                } else {
1330                    totalLinks += normalLinks.size();
1331                }
1332
1333                Log.d(TAG, "refreshLinksList: refresh link list (all): pinned="
1334                    + ", groups=" + (groupedLinks != null ? groupedLinks.size()
1335
1336                if (totalLinks == 0) {
1337                    Log.w(TAG, "refreshLinksList: WARNING! No link data found");
1338                }
1339
1340                // ■■■■■■■adapter■onCreateView■RecyclerView■■■■■■
1341                Log.d(TAG, "refreshLinksList: setting data to adapter directly")
1342                adapter.setPinnedLinks(pinnedLinks);
1343                if (groupedLinks != null) {
1344                    adapter.setGroupedLinks(groupedLinks);
1345                } else {
1346                    // ■■■■■■■■■■■■
1347                    adapter.setFlatLinks(normalLinks);
1348                }
1349                // setGroupedLinks/setFlatLinks■■■■■■■notifyDataSetChanged■■■■■■
1350                Log.d(TAG, "refreshLinksList: data set to adapter, notifyDataSet
1351
1352                // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1353                if (isShuffleMode && recentlyAdded) {
1354                    isShuffleMode = false;
1355                    saveShuffleModeState();
1356                    updateMenuVisibility();
1357                    requireActivity().setTitle(R.string.app_name);
1358                    requireActivity().invalidateOptionsMenu();
1359                }
1360
1361                // ■■■■■■■■■■
1362                RecyclerView recyclerView = binding.recyclerView;
1363                if (recyclerView != null && adapter != null) {
1364                    Log.d(TAG, "refreshLinksList: RecyclerView visibility=" + re
1365                            + ", width=" + recyclerView.getWidth() + ", height="
1366                            + ", adapter itemCount=" + adapter.getItemCount());
1367                }
1368            } else {
1369                // ■■■■■■■■■■■■■
1370                Log.d(TAG, "refreshLinksList: use tag filter logic");
1371                updateContentBySelectedTags();
1372            }
1373            Log.d(TAG, "refreshLinksList: completed");
```

```
1374          } catch (Exception e) {
1375              Log.e(TAG, "refreshLinksList: failed to refresh link list", e);
1376          }
1377      }
1378
1379      // ========== ■■■■■■■■■■■TagsFragment■■■■■■■■■==========
1380
1381      /**
1382       * ■■■■■■■■
1383       * ■■■■■■■■■■■■■■TreeMap■■■■■key■■■■■■■value■
1384       * @param links ■■■■■■■■■
1385       * @return ■■■■■■■Map■■■■■■■■
1386       */
1387      private Map<String, List<LinkItem>> groupLinksByDate(List<LinkItem> links) {
1388          Map<String, List<LinkItem>> groupedLinks = new TreeMap<>(Collections.rev
1389          SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd", Locale.
1390
1391          // ■■■■■■■■■■■■
1392          Collections.sort(links, (a, b) -> Long.compare(b.getTimestamp(), a.getTi
1393
1394          for (LinkItem link : links) {
1395              String date = dateFormat.format(new Date(link.getTimestamp()));
1396              List<LinkItem> dayLinks = groupedLinks.computeIfAbsent(date, k -> ne
1397              dayLinks.add(link);
1398          }
1399
1400          // ■■■■■■■■■■■■■■■■■■■
1401          for (List<LinkItem> dayLinks : groupedLinks.values()) {
1402              Collections.sort(dayLinks, (a, b) -> Long.compare(b.getTimestamp(),
1403          }
1404
1405          return groupedLinks;
1406      }
1407
1408      /**
1409       * ■■■■■■■■■■■■■
1410       * ■■■■■■■■■■■■■■TreeMap■■■■■key■■■■■■■value■
1411       * ■■■■■■■■■■■■■■■■■
1412       * @param links ■■■■■■■■■
1413       * @return ■■■■■■■Map■■■■■■■■
1414       */
1415      private Map<String, List<LinkItem>> groupLinksWithoutSort(List<LinkItem> lin
1416          Map<String, List<LinkItem>> groupedLinks = new TreeMap<>(Collections.rev
1417          SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd", Locale.
1418
1419          for (LinkItem link : links) {
1420              String date = dateFormat.format(new Date(link.getTimestamp()));
1421              List<LinkItem> dayLinks = groupedLinks.computeIfAbsent(date, k -> ne
1422              dayLinks.add(link);
1423          }
```

```
1424
1425          return groupedLinks;
1426      }
1427
1428      /**
1429       * ■■■■■■■■■■■■■■■
1430       * ■■■■■■■■■■■■■■■■■■
1431       * @param filteredLinks ■■■■■■■■■
1432       * @return ■■■■■■■■■■■■■
1433       */
1434      private List<LinkItem> calculatePinnedOverlap(List<LinkItem> filteredLinks)
1435          // ■■■■ id ■■■■■■■■■
1436          Set<Long> filteredIds = new HashSet<>();
1437          for (LinkItem item : filteredLinks) {
1438              filteredIds.add(item.getId());
1439          }
1440
1441          List<LinkItem> pinned = linkDao.getPinnedLinks();
1442          List<LinkItem> pinnedOverlap = new ArrayList<>();
1443          for (LinkItem p : pinned) {
1444              if (filteredIds.contains(p.getId())) {
1445                  pinnedOverlap.add(p);
1446              }
1447          }
1448
1449          return pinnedOverlap;
1450      }
1451
1452      // ========== ■■■■■■■■■ ==========
1453
1454      /**
1455       * ■■■■■■■■
1456       * ■■■■■■■■■■■■■■■■■■
1457       */
1458      private void scrollToDate(RecyclerView recyclerView, String date) {
1459          // ■■■■■■■■■■
1460          int position = adapter.getPositionForDate(date);
1461          if (position != -1) {
1462              recyclerView.post(() -> {
1463                  // ■■ LinearLayoutManager
1464                  LinearLayoutManager layoutManager = (LinearLayoutManager) recycl
1465                  if (layoutManager != null) {
1466                      // ■■■■■■■■■
1467                      int searchBoxHeight = binding.searchEditText.getHeight();
1468                      // ■■■■■■■■■offset ■■■■■■■
1469                      layoutManager.scrollToPositionWithOffset(position, searchBox
1470                  }
1471              });
1472          }
1473      }
```

```
1474
1475      public void onLinkRemarkUpdated(LinkItem item) {
1476          // ■■■■■■■UI■■■■■■■■■
1477          // ■■LinkDao■■■■■■■■■■adapter■■■■■■■■■■■■■■■■■■■■■■■
1478      }
1479
1480      @Override
1481      public void onRequestCustomIcon(LinkItem item, String url) {
1482          Log.d("HomeFragment", "onRequestCustomIcon: " + item.getTitle() + ", URL
1483
1484          // ■■■■■■■
1485          pendingIconItem = item;
1486          pendingIconUrl = url;
1487
1488          // ■■■■■■■■■
1489          Intent intent = ImageUtil.createGalleryPickerIntent();
1490          startActivityForResult(intent, REQUEST_CODE_PICK_ICON);
1491      }
1492
1493      @Override
1494      public void onActivityResult(int requestCode, int resultCode, Intent data) {
1495          super.onActivityResult(requestCode, resultCode, data);
1496
1497          if (requestCode == REQUEST_CODE_PICK_ICON && resultCode == android.app.A
1498              if (data != null && data.getData() != null && pendingIconItem != nul
1499                  Uri selectedImageUri = data.getData();
1500                  Log.d("HomeFragment", "Image selection successful: " + selectedI
1501
1502                  try {
1503                      // 1. ■ URI ■■■ Bitmap
1504                      Bitmap bitmap = ImageUtil.uriToBitmap(requireContext(), sele
1505
1506                      if (bitmap != null) {
1507                          // 2. ■■■■■■■■■■■■■■■■ 256x256■
1508                          Bitmap squareBitmap = ImageUtil.resizeToSquareForShortcu
1509
1510                          // 3. ■■■■■■■
1511                          if (squareBitmap != bitmap) {
1512                              bitmap.recycle();
1513                          }
1514
1515                          // 4. ■■■■■■■
1516                          adapter.createShortcutWithCustomIcon(
1517                              requireContext(),
1518                              pendingIconItem,
1519                              pendingIconUrl,
1520                              squareBitmap
1521                          );
1522
1523                          Log.d("HomeFragment", "Shortcut created successfully (us
```

```
1524                    } else {
1525                        Toast.makeText(requireContext(), "■■■■■■■", Toast.LENGTH_
1526                        Log.e("HomeFragment", "Failed to read image from URI");
1527                    }
1528                } catch (Exception e) {
1529                    Log.e("HomeFragment", "Error processing image: " + e.getMess
1530                    Toast.makeText(requireContext(), "■■■■■■■■: " + e.getMessage(
1531                } finally {
1532                    // ■■■■■■■
1533                    pendingIconItem = null;
1534                    pendingIconUrl = null;
1535                }
1536            } else {
1537                Log.w("HomeFragment", "Image selection failed: data or pendingIc
1538                pendingIconItem = null;
1539                pendingIconUrl = null;
1540            }
1541        }
1542    }
1543
1544    // ========== ■■■■■■■■■■■■■TagsFragment■■■==========
1545
1546    /**
1547     * ■■■■■■■
1548     * ■■■■■■■■■■
1549     */
1550    private void handleTagClick(String tagName) {
1551        if (NO_TAG.equals(tagName)) {
1552            Log.d("HomeFragment", "Clicked no tags");
1553            updateTagSelectionByName(tagName);
1554        } else {
1555            updateTagSelectionByName(tagName);
1556        }
1557    }
1558
1559    /**
1560     * ■■■■■■■■
1561     * ■■■■■■■■■■■■■■■■■■■■■■
1562     * ■■2■■■■■■■■■■■■■■■■■■■■■
1563     */
1564    private void updateTagSelectionByName(String tagName) {
1565        // ■■■ selectedTagNames ■■■■■■■tagID■-1■■■■
1566        cleanInvalidTagsFromSelection();
1567
1568        if (selectedTagNames.contains(tagName)) {
1569            // ■■■■
1570            selectedTagNames.remove(tagName);
1571            Log.d(TAG, "updateTagSelectionByName: unselect tag, tagName=" + tagN
1572            Log.d(TAG, "updateTagSelectionByName: selectedTagNames.size=" + sele
1573            for (String tag : selectedTagNames) {
```

```
1574                Log.d(TAG, "updateTagSelectionByName: tag=" + tag);
1575            }
1576            if (selectedTagNames.isEmpty()) {
1577                // ■■■■■■■■■■■■■■■■■■■HomeFragment■■■■■■
1578                requireActivity().setTitle("■■■■■");
1579                refreshLinksList();  // ■■■■■■■■■
1580                clearSavedSelections();
1581            } else {
1582                // ■■■■■■■■■■■■■■■
1583                updateContentBySelectedTags();
1584            }
1585        } else {
1586            Log.d(TAG, "updateTagSelectionByName: select tag, tagName=" + tagNam
1587
1588            // ■■■■■■■■■■■■■
1589            if (NO_TAG.equals(tagName)) {
1590                // NO_TAG ■■■■■■■■■■■■■■■■■■0■
1591                selectedTagNames.add(tagName);
1592                Log.d(TAG, "updateTagSelectionByName: selectedTagNames.size=" +
1593                updateContentBySelectedTags();
1594            } else {
1595                // ■■■■■■■■■■■
1596                try {
1597                    long tagId = linkDao.getTagIdByName(tagName);
1598                    if (tagId != -1) {
1599                        // ■■■■■■■■■■
1600                        selectedTagNames.add(tagName);
1601                        Log.d(TAG, "updateTagSelectionByName: selectedTagNames.s
1602                        updateContentBySelectedTags();
1603                    } else {
1604                        // ■■■■■■■tagID■-1■■■■■■■
1605                        Log.w(TAG, "updateTagSelectionByName: tag not found (tag
1606                        Toast.makeText(requireContext(), "■■■■■: " + tagName, To
1607                    }
1608                } catch (Exception e) {
1609                    Log.e(TAG, "updateTagSelectionByName: failed to get tagID fo
1610                    Toast.makeText(requireContext(), "■■■■■■: " + tagName, Toast
1611                }
1612            }
1613        }
1614
1615        // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1616        if (isSelectionMode) {
1617            toggleSelectionMode();
1618        }
1619
1620        // ■■■■■■■■■■■■■■■■■■■■
1621        if (tagsAdapter != null) {
1622            tagsAdapter.setSelectedTagNames(selectedTagNames);
1623        }
```

```
1624          if (tagsAdapterCollapsed != null) {
1625              tagsAdapterCollapsed.setSelectedTagNames(selectedTagNames);
1626          }
1627
1628          // ■■■■■■■■■■■■■■■■■■■■■■■■
1629          updateMenuVisibility();
1630      }
1631
1632      /**
1633       * ■■ selectedTagNames ■■■■■■■■tagID■-1■■■■■
1634       * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1635       */
1636      private void cleanInvalidTagsFromSelection() {
1637          if (linkDao == null || selectedTagNames.isEmpty()) {
1638              return;
1639          }
1640
1641          Set<String> invalidTags = new HashSet<>();
1642          boolean needUpdateNoTag = false;
1643
1644          // ■■■■■■■■■■■
1645          for (String tagName : new HashSet<>(selectedTagNames)) {
1646              if (NO_TAG.equals(tagName)) {
1647                  // NO_TAG ■■■■■■■■■■■■■■■■■■count■0■
1648                  // ■■■■■■■■■■ NO_TAG
1649              } else {
1650                  // ■■■■■■■■■■■
1651                  try {
1652                      long tagId = linkDao.getTagIdByName(tagName);
1653                      if (tagId == -1) {
1654                          invalidTags.add(tagName);
1655                          Log.w(TAG, "cleanInvalidTagsFromSelection: found invalid
1656                      }
1657                  } catch (Exception e) {
1658                      invalidTags.add(tagName);
1659                      Log.e(TAG, "cleanInvalidTagsFromSelection: failed to get tag
1660                  }
1661              }
1662          }
1663
1664          // ■■■■■■■
1665          if (!invalidTags.isEmpty()) {
1666              selectedTagNames.removeAll(invalidTags);
1667              Log.d(TAG, "cleanInvalidTagsFromSelection: removed " + invalidTags.s
1668
1669              // ■■ SharedPreferences
1670              SharedPreferences prefs = requireContext().getSharedPreferences(PREF
1671              SharedPreferences.Editor editor = prefs.edit();
1672
1673              // ■■■■■■■■■■■■
```

```
1674              Set<String> validTags = new HashSet<>(selectedTagNames);
1675              validTags.remove(NO_TAG);  // NO_TAG ■■■■
1676
1677              editor.putStringSet(KEY_SELECTED_TAGS, validTags);
1678              if (needUpdateNoTag) {
1679                  editor.putBoolean(KEY_NO_TAG_SELECTED, false);
1680              }
1681              editor.apply();
1682          }
1683      }
1684
1685      // ========== ■■■■■■■■■■■TagsFragment■■■==========
1686
1687      /**
1688       * ■■■■
1689       * ■■■■■■■■■■■■■UI
1690       */
1691      private void loadTags() {
1692          Log.d(TAG, "loadTags: start");
1693          if (tagsAdapter == null || tagsAdapterCollapsed == null) {
1694              Log.e(TAG, "loadTags: tagsAdapter is null, tagsAdapter=" + (tagsAdap
1695                      ", tagsAdapterCollapsed=" + (tagsAdapterCollapsed != null));
1696              return;
1697          }
1698
1699          if (linkDao == null) {
1700              Log.e(TAG, "loadTags: linkDao is null!");
1701              return;
1702          }
1703
1704          // ■■■■■■■■■■■■
1705          new Thread(() -> {
1706              try {
1707                  Log.d(TAG, "loadTags: background thread start");
1708
1709                  // ■■■■Fragment■■■■■■■■■■■■■■■■■■■■■■■■
1710                  // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1711                  if (!isAdded() || linkDao == null) {
1712                      Log.w(TAG, "loadTags: Fragment not attached or linkDao is nu
1713                      return;
1714                  }
1715
1716                  Map<String, Integer> tagsWithCount;
1717                  AtomicInteger noTagCount = new AtomicInteger();
1718                  List<TagsAdapter.TagItem> allTagItems = new ArrayList<>();
1719
1720                  try {
1721                      // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1722                      if (!isAdded() || linkDao == null) {
1723                          Log.w(TAG, "loadTags: Fragment detached or linkDao close
```

```
1724                        return;
1725                    }
1726
1727                    tagsWithCount = linkDao.getTagsWithCount();
1728                    Log.d(TAG, "loadTags: tag count=" + tagsWithCount.size());
1729
1730                    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1731                    // ■■■■■■■■■■■■
1732                    if (!isAdded() || linkDao == null) {
1733                        Log.w(TAG, "loadTags: Fragment detached or linkDao close
1734                        return;
1735                    }
1736                    noTagCount.set(linkDao.getLinksWithoutTags().size());
1737
1738                    // ■■■■■■■■■■■■■tagId■■■■■UI■■■■■■■■■
1739                    // ■■"■■■"■■■■■■■■■■■■■■■0■
1740                    allTagItems.add(new TagsAdapter.TagItem(-1, NO_TAG, noTagCou
1741
1742                    // ■■■■■■■■■■■■■■■■■tagId■
1743                    for (Map.Entry<String, Integer> entry : tagsWithCount.entryS
1744                        // ■■■■■■■■■■
1745                        if (!isAdded() || linkDao == null) {
1746                            Log.w(TAG, "loadTags: Fragment detached or linkDao c
1747                            break;
1748                        }
1749                        long tagId = linkDao.getTagIdByName(entry.getKey());
1750                        if (tagId != -1) {
1751                            allTagItems.add(new TagsAdapter.TagItem(tagId, entry
1752                        }
1753                    }
1754                } catch (IllegalStateException e) {
1755                    Log.w(TAG, "loadTags: database closed, cancel loading", e);
1756                    return;
1757                } catch (Exception e) {
1758                    Log.e(TAG, "loadTags: unexpected exception in background thr
1759                    return;
1760                }
1761
1762                // ■■■■■■■■UI
1763                try {
1764                    // ■■Fragment■■■attached
1765                    if (!isAdded() || getActivity() == null) {
1766                        Log.w(TAG, "loadTags: Fragment not attached or Activity
1767                        return;
1768                    }
1769
1770                    Activity activity = getActivity();
1771                    if (activity != null && !activity.isFinishing()) {
1772                        // ■■final■■■■■■■■■■■lambda■■■■■■■■■■linkDao
1773                        final List<TagsAdapter.TagItem> finalAllTagItems = allTa
```

```
1774                        activity.runOnUiThread(() -> {
1775                            // ■■■■Fragment■■■adapter
1776                            if (!isAdded() || tagsAdapter == null) {
1777                                Log.w(TAG, "loadTags: Fragment state changed dur
1778                                return;
1779                            }
1780
1781                            // ■■■■■■■■■■■■■■■■■■■
1782                            if (isSortMode) {
1783                                if (tagsAdapter != null) {
1784                                    tagsAdapter.setTags(finalAllTagItems);
1785                                    tagsAdapter.setHighlightedTags(highlightedTa
1786                                    tagsAdapter.setSelectedTagNames(selectedTagN
1787                                }
1788                                if (tagsAdapterCollapsed != null) {
1789                                    tagsAdapterCollapsed.setTags(new ArrayList<>
1790                                }
1791                                if (tagsRecyclerViewCollapsed != null) {
1792                                    tagsRecyclerViewCollapsed.setVisibility(View
1793                                }
1794                                if (expandMoreTagsButton != null) {
1795                                    expandMoreTagsButton.setVisibility(View.GONE
1796                                }
1797                            } else {
1798                                // ■■■■■■■■■■■■■■■■
1799                                List<TagsAdapter.TagItem> fixedTags = new ArrayL
1800                                List<TagsAdapter.TagItem> collapsedTags = new Ar
1801
1802                                // ■■■■■■■■■■■■ selectedTagNames■■■■■■■■■■
1803                                // ■■■■■■■■■■■■■■■■■■■■ selectedTagNames■
1804                                try {
1805                                    restoreSelections(false);
1806                                } catch (Exception e) {
1807                                    Log.e(TAG, "loadTags: restoreSelections fail
1808                                }
1809
1810                                // 1) ■■■■■■■■■■■ finalAllTagItems ■■■■ fixedTag
1811                                for (TagsAdapter.TagItem item : finalAllTagItems
1812                                    String name = item.getName() != null ? item.
1813                                    for (String sel : selectedTagNames) {
1814                                        if (sel != null && sel.trim().equals(nam
1815                                            if (!fixedTags.contains(item)) fixed
1816                                            break;
1817                                        }
1818                                    }
1819                                }
1820
1821                                // 2) ■■■■■■■■■■■■■■■■■■■■■■■■■■
1822                                int fixedLimit = FIXED_TAGS_COUNT;
1823                                int addedCount = fixedTags.size();
```

```
1824                                for (int i = 0; i < finalAllTagItems.size() && a
1825                                    TagsAdapter.TagItem item = finalAllTagItems.
1826                                    if (!fixedTags.contains(item)) {
1827                                        fixedTags.add(item);
1828                                        addedCount++;
1829                                        Log.i("FIXED_TAGS NAME", item.getName())
1830                                    }
1831                                }
1832
1833                                // 3) ■■■■■■■■
1834                                for (TagsAdapter.TagItem item : finalAllTagItems
1835                                    if (!fixedTags.contains(item)) {
1836                                        collapsedTags.add(item);
1837                                    }
1838                                }
1839
1840                                if (tagsAdapter != null) {
1841                                    tagsAdapter.setTags(fixedTags);
1842                                    tagsAdapter.setHighlightedTags(highlightedTa
1843                                    tagsAdapter.setSelectedTagNames(selectedTagN
1844                                }
1845
1846                                if (tagsAdapterCollapsed != null) {
1847                                    tagsAdapterCollapsed.setTags(collapsedTags);
1848                                    tagsAdapterCollapsed.setHighlightedTags(high
1849                                    tagsAdapterCollapsed.setSelectedTagNames(sel
1850                                }
1851
1852                                // ■■■■■■■■■■■■■■■■■■■■■■■■■
1853                                if (expandMoreTagsButton != null) {
1854                                    if (collapsedTags.size() > 0) {
1855                                        expandMoreTagsButton.setVisibility(View.
1856                                        updateExpandMoreButtonState();
1857                                    } else {
1858                                        expandMoreTagsButton.setVisibility(View.
1859                                    }
1860                                }
1861
1862                                // ■■■■■■■■■/■■■■■■■■
1863                                if (tagsRecyclerViewCollapsed != null) {
1864                                    tagsRecyclerViewCollapsed.setVisibility(
1865                                        isMoreTagsExpanded && collapsedTags.size
1866                                }
1867                            }
1868
1869                            // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1870                            if (tagsRecyclerView != null) {
1871                                tagsRecyclerView.post(() -> {
1872                                    // ■■■■■■■■■■■■■■
1873                                    if (isSortMode) {
```

```
1874                                    if (toggleTagsButton != null) {
1875                                        toggleTagsButton.setVisibility(View.
1876                                    }
1877                                    return;
1878                                }
1879                                // ■■■■■■■■■■■■■■■■■■■■■
1880                                int height = tagsRecyclerView.getHeight();
1881                                float density = getResources().getDisplayMet
1882                                int collapsedHeightPx = (int) (COLLAPSED_HEI
1883                                if (toggleTagsButton != null) {
1884                                    toggleTagsButton.setVisibility(height >
1885                                }
1886                            });
1887                        }
1888
1889                        // ■■■■■■■■■■■■■■■■■■■■■■■■
1890                        // ■■■■■■■■■■■■■■■■■■■
1891                        if (adapter != null && linkDao != null) {
1892                            if (selectedTagNames.isEmpty()) {
1893                                // ■■■■■■■■■■■■■■■■■■
1894                                refreshLinksList();
1895                            } else {
1896                                Log.d(TAG, "loadTags: selectedTagNames.size=
1897                                // ■■■■■■■■■■■■■■tagID
1898                                for (String tagName : selectedTagNames) {
1899                                    if (NO_TAG.equals(tagName)) {
1900                                        Log.d(TAG, "loadTags: selected tag -
1901                                    } else {
1902                                        try {
1903                                            long tagId = linkDao.getTagIdByN
1904                                            Log.d(TAG, "loadTags: selected t
1905                                        } catch (Exception e) {
1906                                            Log.e(TAG, "loadTags: failed to
1907                                        }
1908                                    }
1909                                }
1910                                // ■■■■■■■■■■■■■■■■■■
1911                                updateContentBySelectedTags();
1912                            }
1913                        }
1914                    });  // ■■runOnUiThread■lambda
1915                } else {
1916                    Log.w(TAG, "loadTags: Activity is null or finished, skip
1917                }
1918            } catch (Exception e) {
1919                Log.e(TAG, "loadTags: runOnUiThread exception", e);
1920            }
1921        } catch (Exception e) {
1922            Log.e(TAG, "loadTags: background thread exception", e);
1923        }
```

```
1924          }).start();
1925      }
1926
1927      // ========== ■■■■■■■■■■■■■■TagsFragment■■■==========
1928
1929      /**
1930       * ■■■■■■■■■■
1931       * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1932       * ■■2■■■■■■■■■
1933       */
1934      private void updateContentBySelectedTags() {
1935          Log.d(TAG, "updateContentBySelectedTags: start, selectedTagNames.size="
1936          try {
1937              if (linkDao == null) {
1938                  Log.e(TAG, "updateContentBySelectedTags: linkDao is null!");
1939                  return;
1940              }
1941              if (adapter == null) {
1942                  Log.e(TAG, "updateContentBySelectedTags: adapter is null!");
1943                  return;
1944              }
1945
1946              // ■■■■■■■■■■■■■■HomeFragment■■■■
1947              if (selectedTagNames.isEmpty()) {
1948                  Log.d(TAG, "updateContentBySelectedTags: no tags selected, use o
1949                  // ■■■■■■■■■■■■■
1950                  List<LinkItem> pinnedLinks = linkDao.getPinnedLinks();
1951                  Map<String, List<LinkItem>> groupedLinks = linkDao.getLinksGroup
1952                  adapter.setPinnedLinks(pinnedLinks);
1953                  adapter.setGroupedLinks(groupedLinks);
1954                  adapter.notifyDataSetChanged();
1955                  try {
1956                      requireActivity().setTitle("■■■■");
1957                  } catch (Exception e) {
1958                      Log.e(TAG, "updateContentBySelectedTags: failed to set title
1959                  }
1960                  return;
1961              }
1962
1963              List<LinkItem> links = new ArrayList<>();
1964              Set<String> tagNames = new HashSet<>();
1965              boolean hasNoTagFilter = false;
1966
1967              // ■■■■■■■■■■
1968              for (String tagName : selectedTagNames) {
1969                  if (NO_TAG.equals(tagName)) {
1970                      hasNoTagFilter = true;
1971                  } else {
1972                      tagNames.add(tagName);
1973                  }
```

```
1974            }
1975
1976            // ████████
1977            if (hasNoTagFilter) {
1978                links.addAll(linkDao.getLinksWithoutTags());
1979            }
1980            if (!tagNames.isEmpty()) {
1981                links.addAll(linkDao.getLinksByTags(tagNames));
1982            }
1983
1984            Log.d(TAG, "updateContentBySelectedTags: filtering completed, links.
1985
1986            // ████
1987            updateTitle(tagNames, hasNoTagFilter);
1988
1989            // ██████████5███
1990            boolean recentlyAdded = (System.currentTimeMillis() - lastLinkAddTim
1991
1992            // ████████████████████████████████
1993            if (isShuffleMode && !recentlyAdded) {
1994                Log.d(TAG, "updateContentBySelectedTags: shuffling " + links.siz
1995                Collections.shuffle(links);
1996            }
1997
1998            // ███████
1999            Map<String, List<LinkItem>> groupedLinks;
2000            if (isShuffleMode && !recentlyAdded) {
2001                // ████████████████
2002                groupedLinks = null;  // ■ null ██████
2003            } else {
2004                // ████████████████
2005                groupedLinks = groupLinksByDate(links);
2006            }
2007            Log.d(TAG, "updateContentBySelectedTags: grouping completed, grouped
2008
2009            // ████████████
2010            List<LinkItem> pinnedOverlap = calculatePinnedOverlap(links);
2011            Log.d(TAG, "updateContentBySelectedTags: pinned overlap=" + pinnedOv
2012
2013            adapter.setPinnedLinks(pinnedOverlap);
2014            if (groupedLinks != null) {
2015                adapter.setGroupedLinks(groupedLinks);
2016            } else {
2017                // ████████████
2018                adapter.setFlatLinks(links);
2019            }
2020            adapter.notifyDataSetChanged();
2021
2022            // ██████
2023            saveSelections(tagNames, hasNoTagFilter);
```

```
2024              Log.d(TAG, "updateContentBySelectedTags: completed");
2025          } catch (Exception e) {
2026              Log.e(TAG, "updateContentBySelectedTags: exception occurred", e);
2027          }
2028      }
2029
2030      /**
2031       * ■■■■
2032       * ■■■■■■■■■■Activity■■
2033       */
2034      private void updateTitle(Set<String> tags, boolean includeNoTag) {
2035          try {
2036              StringBuilder title = new StringBuilder();
2037              if (tags.isEmpty() && !includeNoTag) {
2038                  title.append("■■■■");
2039              } else {
2040                  if (includeNoTag) {
2041                      title.append("■■■");
2042                      if (!tags.isEmpty()) {
2043                          title.append(" + ");
2044                      }
2045                  }
2046                  if (!tags.isEmpty()) {
2047                      title.append(String.join(" + ", tags));
2048                  }
2049              }
2050              requireActivity().setTitle(title.toString());
2051              Log.d(TAG, "updateTitle: " + title.toString());
2052          } catch (Exception e) {
2053              Log.e(TAG, "updateTitle: failed to update title", e);
2054          }
2055      }
2056
2057      /**
2058       * ■■■■■■■
2059       * ■■■■■■■■■■SharedPreferences
2060       */
2061      private void saveSelections(Set<String> tags, boolean includeNoTag) {
2062          SharedPreferences.Editor editor = requireContext()
2063                  .getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
2064                  .edit();
2065
2066          // ■■■■■■■
2067          editor.putStringSet(KEY_SELECTED_TAGS, tags);
2068          editor.putBoolean(KEY_NO_TAG_SELECTED, includeNoTag);
2069          editor.apply();
2070      }
2071
2072      /**
2073       * ■■■■■■■■■■
```

```
2074       */
2075      private void clearSavedSelections() {
2076          requireContext().getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
2077                  .edit()
2078                  .clear()
2079                  .commit();
2080      }
2081
2082      /**
2083       * ■■■■■■■■■■■■■■■■■■■■■■
2084       * ■SharedPreferences■■■■■■■■■■■■■■■■■■
2085       * ■■onCreateView■■■■■■■■■■■■■■■■■
2086       */
2087      private void restoreSelectionsSync() {
2088          try {
2089              Log.d(TAG, "restoreSelectionsSync: start");
2090              SharedPreferences prefs = requireContext().getSharedPreferences(PREF
2091              Set<String> savedTags = prefs.getStringSet(KEY_SELECTED_TAGS, new Ha
2092              boolean noTagSelected = prefs.getBoolean(KEY_NO_TAG_SELECTED, false)
2093
2094              Log.d(TAG, "restoreSelectionsSync: savedTags.size=" + savedTags.size
2095
2096              // ■■■■■■■
2097              selectedTagNames.clear();
2098              if (noTagSelected) {
2099                  selectedTagNames.add(NO_TAG);
2100              }
2101              selectedTagNames.addAll(savedTags);
2102
2103              Log.d(TAG, "restoreSelectionsSync: after restore selectedTagNames.si
2104          } catch (Exception e) {
2105              Log.e(TAG, "restoreSelectionsSync: failed to restore selection state
2106          }
2107      }
2108
2109      /**
2110       * ■■■■■■■
2111       * ■SharedPreferences■■■■■■■■■■■■■UI■■■■
2112       * @param shouldLoadData ■■■■■■■■true■■■■■■■false■■■■■UI■■■■
2113       */
2114      private void restoreSelections(boolean shouldLoadData) {
2115          try {
2116              Log.d(TAG, "restoreSelections: start, shouldLoadData=" + shouldLoadD
2117              SharedPreferences prefs = requireContext().getSharedPreferences(PREF
2118              Set<String> savedTags = prefs.getStringSet(KEY_SELECTED_TAGS, new Ha
2119              boolean noTagSelected = prefs.getBoolean(KEY_NO_TAG_SELECTED, false)
2120
2121              Log.d(TAG, "restoreSelections: savedTags.size=" + savedTags.size() +
2122
2123              // ■■■■■■■
```

```
2124                selectedTagNames.clear();
2125                if (noTagSelected) {
2126                    selectedTagNames.add(NO_TAG);
2127                }
2128                selectedTagNames.addAll(savedTags);
2129
2130                Log.d(TAG, "restoreSelections: after restore selectedTagNames.size="
2131
2132                // ■■■■■■■■■■■■■■■■■■■■
2133                if (tagsAdapter != null) {
2134                    tagsAdapter.setSelectedTagNames(selectedTagNames);
2135                }
2136                if (tagsAdapterCollapsed != null) {
2137                    tagsAdapterCollapsed.setSelectedTagNames(selectedTagNames);
2138                }
2139
2140                // ■■■■■■■■■■■■■■■■■■
2141                if (shouldLoadData) {
2142                    // ■■adapter■linkDao■■■■■
2143                    if (adapter == null || linkDao == null) {
2144                        Log.w(TAG, "restoreSelections: adapter or linkDao not initia
2145                        return;
2146                    }
2147
2148                    if (selectedTagNames.isEmpty()) {
2149                        // ■■■■■■■■■■■■■■■■■■■■■■
2150                        // ■■onViewCreated■■■■■■■■■■■■■■■■■■■■■■UI■■■■■
2151                        Log.d(TAG, "restoreSelections: no tags selected, keep existi
2152                        ///■■■refreshLinksList()■■■■■■■■■■■
2153                    } else {
2154                        // ■■■■■■■■■■■■■■■■■■■■■■■■■■■
2155                        Log.d(TAG, "restoreSelections: filter content by selected ta
2156                        updateContentBySelectedTags();
2157                    }
2158                }
2159
2160                Log.d(TAG, "restoreSelections: completed");
2161            } catch (Exception e) {
2162                Log.e(TAG, "restoreSelections: failed to restore selection state", e
2163                // ■■■■■■■■■■■■■■■■■■■■■■■■■■■
2164                if (shouldLoadData && selectedTagNames.isEmpty() && adapter != null
2165                    Log.d(TAG, "restoreSelections: restore failed, try to show all l
2166                    refreshLinksList();
2167                }
2168            }
2169        }
2170
2171        /**
2172         * ■■■■■■■■■■■■■■■
2173         * ■SharedPreferences■■■■■■■■
```

```
2174       */
2175      private void restoreSelections() {
2176          restoreSelections(true);
2177      }
2178
2179      // ========== ■■■■■UI■■■■■■TagsFragment■■■==========
2180
2181      /**
2182       * ■■■■■■■■/■■
2183       */
2184      private void toggleMoreTagsExpansion() {
2185          isMoreTagsExpanded = !isMoreTagsExpanded;
2186          updateExpandMoreButtonState();
2187
2188          // ■■/■■■■■■■
2189          if (tagsAdapterCollapsed != null && tagsRecyclerViewCollapsed != null) {
2190              List<TagsAdapter.TagItem> collapsedTags = tagsAdapterCollapsed.getTa
2191              boolean shouldShowCollapsed = isMoreTagsExpanded && collapsedTags.si
2192              tagsRecyclerViewCollapsed.setVisibility(shouldShowCollapsed ? View.V
2193          }
2194
2195          // ■■■■■■■■
2196          updateTagsScrollViewHeight();
2197      }
2198
2199      /**
2200       * ■■■■■■■■■■
2201       */
2202      private void updateExpandMoreButtonState() {
2203          if (textExpandMore != null && iconExpandMore != null) {
2204              if (isMoreTagsExpanded) {
2205                  textExpandMore.setText("■■■■");
2206                  iconExpandMore.setImageResource(R.drawable.ic_expand_less);
2207                  iconExpandMore.setContentDescription("■■");
2208              } else {
2209                  textExpandMore.setText("■■■■■■");
2210                  iconExpandMore.setImageResource(R.drawable.ic_expand_more);
2211                  iconExpandMore.setContentDescription("■■");
2212              }
2213          }
2214      }
2215
2216      /**
2217       * ■■■■■■■■■■■
2218       */
2219      private void updateTagsScrollViewHeight() {
2220          if (tagsScrollView != null) {
2221              ViewGroup.LayoutParams params = tagsScrollView.getLayoutParams();
2222              float density = getResources().getDisplayMetrics().density;
2223
```

```
2224            // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
2225            if (isMoreTagsExpanded || isSortMode) {
2226                // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
2227                // ■■■■■■■■■■■■ - ■■■■■■ - ■■■■/■■■■ - ■■■■■
2228                int screenHeight = getResources().getDisplayMetrics().heightPixe
2229
2230                // ■■■■■■■■■■■■■■■
2231                int searchBoxHeight = 0;
2232                if (searchEditText != null && searchEditText.getVisibility() ==
2233                    searchEditText.measure(
2234                        View.MeasureSpec.makeMeasureSpec(0, View.MeasureSpec.UNS
2235                        View.MeasureSpec.makeMeasureSpec(0, View.MeasureSpec.UNS
2236                    searchBoxHeight = searchEditText.getMeasuredHeight();
2237                }
2238
2239                // ■■■■■■■■■■■■■■■ + ■■padding + ■■■■■■
2240                // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
2241                int statusBarHeight = 0;
2242                int resourceId = getResources().getIdentifier("status_bar_height
2243                if (resourceId > 0) {
2244                    statusBarHeight = getResources().getDimensionPixelSize(resou
2245                }
2246
2247                int bottomMargin = (int) (64 * density);  // ■■■■■■■■■■■■■■
2248                int availableHeight = screenHeight - searchBoxHeight - statusBar
2249
2250                // ■■■■■■■■60%■■■400dp■■■■■■
2251                int maxHeightByScreen = (int) (availableHeight * 0.6f);
2252                int maxHeightByDp = (int) (SORT_MODE_MAX_HEIGHT_DP * density);
2253                int maxHeightPx = Math.min(maxHeightByScreen, maxHeightByDp);
2254
2255                // ■■■■■■■■
2256                tagsScrollView.measure(
2257                    View.MeasureSpec.makeMeasureSpec(params.width, View.MeasureS
2258                    View.MeasureSpec.makeMeasureSpec(0, View.MeasureSpec.UNSPECI
2259                int measuredHeight = tagsScrollView.getMeasuredHeight();
2260
2261                // ■■■■■■■■■■■■■■■■■ wrap_content■■■■■■■■■■■■■■■■■
2262                params.height = measuredHeight <= maxHeightPx
2263                    ? ViewGroup.LayoutParams.WRAP_CONTENT
2264                    : maxHeightPx;
2265
2266                String mode = isSortMode ? "■■■■■" : "■■■■■■■■■";
2267                Log.d(TAG, "updateTagsScrollViewHeight: " + mode + ", screenHeig
2268                    + ", searchBoxHeight=" + searchBoxHeight + ", availableHeigh
2269                    + ", measuredHeight=" + measuredHeight + ", maxHeightPx=" +
2270                    + ", ■■■■■=" + params.height);
2271            } else {
2272                // ■■■■■■■■■■■■■■■■ 120dp
2273                int maxHeightPx = (int) (COLLAPSED_HEIGHT_DP * density);
```

```
2274                // ████████
2275                tagsScrollView.measure(
2276                    View.MeasureSpec.makeMeasureSpec(params.width, View.MeasureS
2277                    View.MeasureSpec.makeMeasureSpec(0, View.MeasureSpec.UNSPECI
2278                int measuredHeight = tagsScrollView.getMeasuredHeight();
2279                // ████████████████ wrap_content██████████
2280                params.height = measuredHeight <= maxHeightPx
2281                    ? ViewGroup.LayoutParams.WRAP_CONTENT
2282                    : maxHeightPx;
2283            }
2284            tagsScrollView.setLayoutParams(params);
2285        }
2286    }
2287
2288    /**
2289     * █████████/██
2290     */
2291    private void toggleTagsExpansion() {
2292        isTagsExpanded = !isTagsExpanded;
2293
2294        if (tagsScrollView == null || arrowIndicator == null) return;
2295
2296        ViewGroup.LayoutParams params = tagsScrollView.getLayoutParams();
2297
2298        if (isTagsExpanded) {
2299            // ████ - ██████████
2300            params.height = ViewGroup.LayoutParams.WRAP_CONTENT;
2301            arrowIndicator.setImageResource(R.drawable.ic_expand_less);
2302            arrowIndicator.setContentDescription("████");
2303        } else {
2304            // ████ - ██████
2305            float density = getResources().getDisplayMetrics().density;
2306            params.height = (int) (COLLAPSED_HEIGHT_DP * density);
2307            arrowIndicator.setImageResource(R.drawable.ic_expand_more);
2308            arrowIndicator.setContentDescription("████");
2309        }
2310
2311        tagsScrollView.setLayoutParams(params);
2312
2313        // ███████████████
2314        if (toggleTagsButton != null) {
2315            toggleTagsButton.bringToFront();
2316        }
2317    }
2318
2319    /**
2320     * ████████████████
2321     */
2322    private void checkTagsVisibility() {
2323        if (tagsRecyclerView == null || toggleTagsButton == null) return;
```

```
2324
2325          // ■■■■■■■■■■
2326          if (isSortMode) {
2327              toggleTagsButton.setVisibility(View.GONE);
2328              return;
2329          }
2330
2331          // ■■■■
2332          int height = tagsRecyclerView.getHeight();
2333          // ■■■■■■■■■■
2334          float density = getResources().getDisplayMetrics().density;
2335          int collapsedHeightPx = (int) (COLLAPSED_HEIGHT_DP * density);
2336
2337          // ■■■■■■■■■■■■■■■■■■■■■■■
2338          toggleTagsButton.setVisibility(height > collapsedHeightPx ? View.VISIBLE
2339
2340          // ■■■■■■■■■■■■■■ isTagsExpanded ■■
2341          if (toggleTagsButton.getVisibility() == View.VISIBLE && arrowIndicator !
2342              arrowIndicator.setImageResource(isTagsExpanded ?
2343                  R.drawable.ic_expand_less : R.drawable.ic_expand_more);
2344          }
2345      }
2346
2347      /**
2348       * ■■■■■■■■■
2349       */
2350      private void loadHighlightedTags() {
2351          SharedPreferences prefs = requireContext().getSharedPreferences(PREF_NAM
2352          highlightedTags = new HashSet<>(prefs.getStringSet(PREF_HIGHLIGHTED_TAGS
2353      }
2354
2355      /**
2356       * ■■■■■■■
2357       */
2358      private void toggleTagHighlight(String tagName) {
2359          // ■■■■■■■■■■■■■■■■
2360          if (highlightedTags.contains(tagName)) {
2361              highlightedTags.remove(tagName);
2362          } else {
2363              // ■■■■■■■
2364              highlightedTags.add(tagName);
2365          }
2366
2367          // ■■■SharedPreferences
2368          SharedPreferences.Editor editor = requireContext()
2369                  .getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
2370                  .edit();
2371          editor.putStringSet(PREF_HIGHLIGHTED_TAGS, highlightedTags);
2372          editor.apply();
2373
```

```
2374          // ■■■■■■■
2375          if (tagsAdapter != null) {
2376              tagsAdapter.setHighlightedTags(highlightedTags);
2377          }
2378          if (tagsAdapterCollapsed != null) {
2379              tagsAdapterCollapsed.setHighlightedTags(highlightedTags);
2380          }
2381
2382          // ■■■■
2383          String message = highlightedTags.contains(tagName) ? "■■■■■■" + tagName
2384          Snackbar.make(requireView(), message, Snackbar.LENGTH_SHORT).show();
2385      }
2386
2387      /**
2388       * ■■■■■■■■■■
2389       */
2390      private void saveTagOrderToDatabase() {
2391          List<Long> orderedTagIds = new ArrayList<>();
2392
2393          // ■■■■■■■■■■■■■■■■■
2394          // ■■■■■■■■■■■■■■■■■■■■■■
2395          List<TagsAdapter.TagItem> allTags = new ArrayList<>();
2396          if (tagsAdapter != null) {
2397              allTags.addAll(tagsAdapter.getTags());
2398          }
2399          if (tagsAdapterCollapsed != null && !isSortMode) {
2400              allTags.addAll(tagsAdapterCollapsed.getTags());
2401          }
2402
2403          for (TagsAdapter.TagItem tag : allTags) {
2404              orderedTagIds.add(tag.getId());
2405          }
2406
2407          linkDao.saveTagOrder(orderedTagIds);
2408      }
2409
2410      // ========== ■■■■■■■■■■■■TagsFragment■■■==========
2411
2412      /**
2413       * ■■■■■■■■■■
2414       */
2415      private void showTagOptionsDialog(String tag) {
2416          String[] options = {"■■■■■", "■■■■■■■■■■■■", "■■■■■■", "■■■■■"};
2417
2418          new AlertDialog.Builder(requireContext())
2419              .setTitle("■■■■■")
2420              .setItems(options, (dialog, which) -> {
2421                  switch (which) {
2422                      case 0: // ■■■■■
2423                          confirmDeleteTag(tag, false);
```

```
                        break;
                    case 1: // ■■■■■■■■■■■■
                        confirmDeleteTag(tag, true);
                        break;
                    case 2: // ■■■■■
                        publishTagToWebsite(tag);
                        break;
                    case 3: // ■■■■
                        toggleTagHighlight(tag);
                        break;
                }
            })
            .show();
    }

    /**
     * ■■■■■■■
     */
    private void confirmDeleteTag(String tag, boolean isCascading) {
        try {
            new AlertDialog.Builder(requireContext())
                .setTitle("■■■■")
                .setMessage("■■■■■■■■ \"" + tag + "\" ■■")
                .setPositiveButton("■■", (dialog, which) -> {
                    try {
                        Log.d("HomeFragment", "Start deleting tag: " + tag);

                        // ■■■■■■■■■■
                        if (!isCascading) {
                            linkDao.deleteTag(tag);
                        } else { // ■■■■■
                            linkDao.deleteTagWithLinks(tag);
                        }
                        Log.d("HomeFragment", "Tag deleted from database");

                        // ■■■■■■■■■■■■■■■
                        selectedTagNames.remove(tag);
                        Log.d("HomeFragment", "Tag removed from selected set");

                        // ■■■■■■■
                        loadTags();
                        Log.d("HomeFragment", "Tag list reloaded");

                        // ■■■■■■■
                        if (selectedTagNames.isEmpty()) {
                            refreshLinksList();  // ■■■■■■■■
                        } else {
                            updateContentBySelectedTags();
                        }

```

```
2474                        // ■■■■■■
2475                        Toast.makeText(requireContext(),
2476                            "■■■■■",
2477                            Toast.LENGTH_SHORT).show();
2478
2479                    } catch (Exception e) {
2480                        Log.e("HomeFragment", "Error deleting tag", e);
2481                        Toast.makeText(requireContext(),
2482                            "■■■■■■: " + e.getMessage(),
2483                            Toast.LENGTH_SHORT).show();
2484                    }
2485                })
2486                .setNegativeButton("■■", null)
2487                .show();
2488        } catch (Exception e) {
2489            Log.e("HomeFragment", "Error showing confirmation dialog", e);
2490            Toast.makeText(requireContext(),
2491                "■■■■: " + e.getMessage(),
2492                Toast.LENGTH_SHORT).show();
2493        }
2494
2495        // ■■■■■■■
2496        if (tagsRecyclerView != null) {
2497            tagsRecyclerView.post(this::checkTagsVisibility);
2498        }
2499    }
2500
2501    /**
2502     * ■■■■■■■
2503     * ■■■■■■■■■■0■■■■■■■■■■Sync■■■■■■■■■■■■
2504     */
2505    private void publishTagToWebsite(String tag) {
2506        Log.d("HomeFragment", "Start publishing tag to website: " + tag);
2507        // TODO: ■■■■■■■■■■■■■Sync■■
2508        Snackbar.make(requireView(), "■■■■■■■■■Sync■■", Snackbar.LENGTH_SHORT).sh
2509    }
2510
2511    /**
2512     * ■■■■■■■■■■
2513     */
2514    private void showAddTagDialog() {
2515        View dialogView = getLayoutInflater().inflate(R.layout.dialog_add_tag, n
2516        EditText tagInput = dialogView.findViewById(R.id.edit_tag_input);
2517
2518        new AlertDialog.Builder(requireContext())
2519                .setTitle("■■■■■")
2520                .setView(dialogView)
2521                .setPositiveButton("■■", (dialog, which) -> {
2522                    String tagName = tagInput.getText().toString().trim();
2523                    if (!TextUtils.isEmpty(tagName)) {
```

```
                        // ■■■■■■■■■
                        List<String> existingTags = linkDao.getAllTags();
                        if (existingTags.contains(tagName)) {
                            // ■■■■■■■■■■■■
                            Snackbar.make(requireView(), "■■■■■■", Snackbar.LENGT
                            return;
                        }

                        // ■■■■■■
                        long tagId = linkDao.addTag(tagName);
                        if (tagId != -1) {
                            // ■■■■■■■■■
                            loadTags();
                            // ■■■■■
                            Snackbar.make(requireView(), "■■■■■■■" + tagName, Sna
                        }
                    }
                })
                .setNegativeButton("■■", null)
                .show();
    }
}
```

```
// MISSING: person/notfresh/readingshare/adapter/LinkAdapter.java
package person.notfresh.readingshare.db;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.util.Log;
import android.text.TextUtils;

import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.Locale;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Set;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;

import org.json.JSONArray;
import org.json.JSONException;

import person.notfresh.readingshare.model.LinkItem;
```

```
28
29  public class LinkDao {
30      private LinkDbHelper dbHelper;
31      private SQLiteDatabase database;
32      private static final String TAG = "LinkDao";
33
34      public LinkDao(Context context) {
35          dbHelper = new LinkDbHelper(context);
36      }
37
38      public LinkDao(Context context, String databaseName) {
39          dbHelper = new LinkDbHelper(context, databaseName);
40      }
41
42      public void open() {
43          database = dbHelper.getWritableDatabase();
44      }
45
46      public void close() {
47          dbHelper.close();
48      }
49
50      public long insertLink(LinkItem item) {
51          ContentValues values = new ContentValues();
52          values.put(LinkDbHelper.COLUMN_TITLE, item.getTitle());
53          values.put(LinkDbHelper.COLUMN_URL, item.getUrl());
54          values.put(LinkDbHelper.COLUMN_SOURCE_APP, item.getSourceApp());
55          values.put(LinkDbHelper.COLUMN_TIMESTAMP, item.getTimestamp());
56          values.put(LinkDbHelper.COLUMN_ORIGINAL_INTENT, item.getOriginalIntent()
57          values.put(LinkDbHelper.COLUMN_TARGET_ACTIVITY, item.getTargetActivity()
58          values.put(LinkDbHelper.COLUMN_REMARK, item.getRemark());
59          values.put(LinkDbHelper.COLUMN_SUMMARY, item.getSummary());
60
61          long linkId = database.insert(LinkDbHelper.TABLE_LINKS, null, values);
62          item.setId(linkId);
63          updateLinkTags(item);
64          return linkId;
65      }
66
67      public void deleteLink(String url) {
68          database.delete(
69                  LinkDbHelper.TABLE_LINKS,
70                  LinkDbHelper.COLUMN_URL + " = ?",
71                  new String[]{url}
72          );
73      }
74
75      /**
76       * ■■ URL ■■■■■■■■■■
77       * @param url ■■■■ URL
```

```
 78       * @return ■■ URL ■■■■■ true■■■■■ false
 79       */
 80      public boolean urlExists(String url) {
 81          Cursor cursor = database.query(
 82                  LinkDbHelper.TABLE_LINKS,
 83                  new String[]{LinkDbHelper.COLUMN_ID},
 84                  LinkDbHelper.COLUMN_URL + " = ?",
 85                  new String[]{url},
 86                  null,
 87                  null,
 88                  null,
 89                  "1"  // ■■■■■■■■■
 90          );
 91          boolean exists = cursor.getCount() > 0;
 92          cursor.close();
 93          return exists;
 94      }
 95
 96      /**
 97       * ■■■■■■■■■■■■■■■■■
 98       */
 99      public boolean deleteLink(long linkId) {
100          Log.d("LinkDao", "■■■■ID: " + linkId);
101          SQLiteDatabase db = dbHelper.getWritableDatabase();
102          boolean success = false;
103
104          db.beginTransaction();
105          try {
106              // ■■■■■■■■■■
107              int linkTagsDeleted = db.delete(
108                  LinkDbHelper.TABLE_LINK_TAGS,
109                  LinkDbHelper.COLUMN_LINK_ID + " = ?",
110                  new String[]{String.valueOf(linkId)}
111              );
112              Log.d("LinkDao", "■■■ " + linkTagsDeleted + " ■■■■■");
113
114              // ■■■■■■
115              int linksDeleted = db.delete(
116                  LinkDbHelper.TABLE_LINKS,
117                  LinkDbHelper.COLUMN_ID + " = ?",
118                  new String[]{String.valueOf(linkId)}
119              );
120              Log.d("LinkDao", "■■■ " + linksDeleted + " ■■■■■");
121
122              db.setTransactionSuccessful();
123              success = (linksDeleted > 0);
124          } finally {
125              db.endTransaction();
126          }
127
```

```
128         return success;
129     }
130
131     public void updateLinkTitle(String url, String newTitle) {
132         ContentValues values = new ContentValues();
133         values.put(LinkDbHelper.COLUMN_TITLE, newTitle);
134
135         database.update(
136                 LinkDbHelper.TABLE_LINKS,
137                 values,
138                 LinkDbHelper.COLUMN_URL + " = ?",
139                 new String[]{url}
140         );
141     }
142
143     public void togglePinStatus(long linkId) {
144         SQLiteDatabase db = dbHelper.getWritableDatabase();
145         Log.d("LinkDao", "■■■■■■■■, linkId: " + linkId);
146
147         Cursor cursor = db.query(LinkDbHelper.TABLE_LINKS, new String[]{"is_pinn
148                 "_id = ?", new String[]{String.valueOf(linkId)}, null, null, nul
149
150         int currentStatus = 0;
151         if (cursor.moveToFirst()) {
152             currentStatus = cursor.getInt(0);
153             Log.d("LinkDao", "■■■■■■: " + currentStatus);
154         }
155         cursor.close();
156
157         ContentValues values = new ContentValues();
158         values.put("is_pinned", currentStatus == 0 ? 1 : 0);
159
160         int updatedRows = db.update(LinkDbHelper.TABLE_LINKS, values, "_id = ?",
161                 new String[]{String.valueOf(linkId)});
162         Log.d("LinkDao", "■■■■: " + updatedRows + " ■■■■");
163     }
164
165     public void updateSummary(long linkId, String summary) {
166         ContentValues values = new ContentValues();
167         values.put(LinkDbHelper.COLUMN_SUMMARY, summary);
168         database.update(LinkDbHelper.TABLE_LINKS, values,
169                 LinkDbHelper.COLUMN_ID + " = ?",
170                 new String[]{String.valueOf(linkId)});
171     }
172
173     public void updateClickCount(long id, int count) {
174         SQLiteDatabase db = dbHelper.getWritableDatabase();
175         db.beginTransaction();
176         try {
177             ContentValues values = new ContentValues();
```

```
178             values.put("click_count", count);
179             db.update("links", values, "_id = ?", new String[]{String.valueOf(id
180             db.setTransactionSuccessful();
181         } finally {
182             db.endTransaction();
183         }
184     }
185 
186     public void _________(){}
187 
188     public List<LinkItem> getAllLinks() {
189         List<LinkItem> links = new ArrayList<>();
190 
191         Cursor cursor = database.query(
192                 LinkDbHelper.TABLE_LINKS,
193                 null,
194                 null,
195                 null,
196                 null,
197                 null,
198                 LinkDbHelper.COLUMN_TIMESTAMP + " DESC"
199         );
200 
201         if (cursor.moveToFirst()) {
202             do {
203                 long id = cursor.getLong(cursor.getColumnIndexOrThrow(LinkDbHelp
204                 String title = cursor.getString(cursor.getColumnIndexOrThrow(Lin
205                 String url = cursor.getString(cursor.getColumnIndexOrThrow(LinkD
206                 String summary = cursor.getString(cursor.getColumnIndexOrThrow(L
207                 String remark = cursor.getString(cursor.getColumnIndexOrThrow(Li
208                 String sourceApp = cursor.getString(cursor.getColumnIndexOrThrow
209                 String originalIntent = cursor.getString(cursor.getColumnIndexOr
210                 String targetActivity = cursor.getString(cursor.getColumnIndexOr
211                 long timestamp = cursor.getLong(cursor.getColumnIndexOrThrow(Lin
212                 int clickCount = cursor.getInt(cursor.getColumnIndexOrThrow("cli
213 
214                 LinkItem item = new LinkItem(title, url, sourceApp, originalInte
215                 item.setId(id);  // ■■ id
216                 item.setSummary(summary);
217                 item.setRemark(remark);
218                 item.setClickCount(clickCount);  // ■■ clickCount
219 
220                 // ■■■■■■■■
221                 List<String> tags = getLinkTags(id);
222                 for (String tag : tags) {
223                     item.addTag(tag);
224                 }
225 
226                 links.add(item);
227             } while (cursor.moveToNext());
```

```
228         }
229         cursor.close();
230 
231         return links;
232     }
233 
234     // ■■■■■■■■■■
235     public Map<String, List<LinkItem>> getLinksGroupByDate() {
236         Map<String, List<LinkItem>> groupedLinks = new TreeMap<>(Collections.rev
237         SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd", Locale.
238 
239         List<LinkItem> allLinks = getAllLinks();
240         for (LinkItem link : allLinks) {
241             String date = dateFormat.format(new Date(link.getTimestamp()));
242             groupedLinks.computeIfAbsent(date, k -> new ArrayList<>()).add(link)
243         }
244 
245         return groupedLinks;
246     }
247 
248     public List<LinkItem> getLinksWithoutTags() {
249         List<LinkItem> links = new ArrayList<>();
250 
251         // ■■■■■■■■■■■■■■■■■■■■■■
252         String query = "SELECT * FROM " + LinkDbHelper.TABLE_LINKS + " l " +
253                 "WHERE NOT EXISTS (SELECT 1 FROM " + LinkDbHelper.TABLE_LINK_TAG
254                 " lt WHERE l." + LinkDbHelper.COLUMN_ID + " = lt." + LinkDbHelpe
255                 "ORDER BY " + LinkDbHelper.COLUMN_TIMESTAMP + " DESC";
256 
257         Cursor cursor = database.rawQuery(query, null);
258         if (cursor.moveToFirst()) {
259             do {
260                 LinkItem item = createLinkItemFromCursor(cursor);
261                 links.add(item);
262             } while (cursor.moveToNext());
263         }
264         cursor.close();
265         return links;
266     }
267 
268     public List<LinkItem> getLinksByTag(String tag) {
269         List<LinkItem> links = new ArrayList<>();
270         String query = "SELECT DISTINCT l.* FROM " + LinkDbHelper.TABLE_LINKS +
271                 "JOIN " + LinkDbHelper.TABLE_LINK_TAGS + " lt ON l." + LinkDbHel
272                 "JOIN " + LinkDbHelper.TABLE_TAGS + " t ON lt." + LinkDbHelper.C
273                 "WHERE t." + LinkDbHelper.COLUMN_TAG_NAME + " = ?";
274 
275         Cursor cursor = database.rawQuery(query, new String[]{tag});
276 
277         if (cursor.moveToFirst()) {
```

```
278             do {
279                 long id = cursor.getLong(cursor.getColumnIndexOrThrow(LinkDbHelp
280                 String title = cursor.getString(cursor.getColumnIndexOrThrow(Lin
281                 String url = cursor.getString(cursor.getColumnIndexOrThrow(LinkD
282                 String sourceApp = cursor.getString(cursor.getColumnIndexOrThrow
283                 String originalIntent = cursor.getString(cursor.getColumnIndexOr
284                 String targetActivity = cursor.getString(cursor.getColumnIndexOr
285 
286                 LinkItem item = new LinkItem(title, url, sourceApp, originalInte
287                 item.setId(id);
288 
289                 // ■■■■■■■■■■
290                 List<String> tags = getLinkTags(id);
291                 for (String t : tags) {
292                     item.addTag(t);
293                 }
294 
295                 links.add(item);
296             } while (cursor.moveToNext());
297         }
298         cursor.close();
299 
300         return links;
301     }
302 
303     public List<LinkItem> getLinksByTags(Set<String> tags) {
304         List<LinkItem> links = new ArrayList<>();
305         if(tags.isEmpty()){
306             return links;
307         }
308         // ■■■■■■■■■■■■■■■■
309         String query = "SELECT DISTINCT l.* FROM " + LinkDbHelper.TABLE_LINKS +
310                 "JOIN " + LinkDbHelper.TABLE_LINK_TAGS + " lt ON l." + LinkDbHel
311                 "JOIN " + LinkDbHelper.TABLE_TAGS + " t ON lt." + LinkDbHelper.C
312                 "WHERE t." + LinkDbHelper.COLUMN_TAG_NAME + " IN (" + makePlaceh
313                 "ORDER BY l." + LinkDbHelper.COLUMN_TIMESTAMP + " DESC";
314 
315         String[] selectionArgs = tags.toArray(new String[0]);
316         Cursor cursor = database.rawQuery(query, selectionArgs);
317 
318         if (cursor.moveToFirst()) {
319             do {
320                 LinkItem item = createLinkItemFromCursor(cursor);
321                 links.add(item);
322             } while (cursor.moveToNext());
323         }
324         cursor.close();
325         return links;
326     }
327 
```

```
328      public List<LinkItem> getPinnedLinks() {
329          List<LinkItem> pinnedLinks = new ArrayList<>();
330          SQLiteDatabase db = dbHelper.getReadableDatabase();
331
332          Log.d("LinkDao", "■■■■■■■");
333          Cursor cursor = db.query(LinkDbHelper.TABLE_LINKS, null, "is_pinned = 1"
334                  null, null, "timestamp DESC");
335
336          Log.d("LinkDao", "■■ " + cursor.getCount() + " ■■■■■■");
337          if (cursor.moveToFirst()) {
338              do {
339                  LinkItem item = cursorToLinkItem(cursor);
340                  List<String> tags = getLinkTags(item.getId());
341                  for (String tag : tags) {
342                      item.addTag(tag);
343                  }
344                  pinnedLinks.add(item);
345              } while (cursor.moveToNext());
346          }
347          cursor.close();
348
349          return pinnedLinks;
350      }
351
352
353
354      public void ________________________________________(){}
355      ////////////////////////////////////////////////////////////////////////////
356      ////////////////////////////////////////////////////////////////////////////
357      ////////////////////////////////////////////////////////////////////////////
358      ////////////////////////////////////////////////////////////////////////////
359      ////////////////////////////////////////////////////////////////////////////
360      ////////////////////////////////////////////////////////////////////////////
361      ////////////////////////////////////////////////////////////////////////////
362
363
364
365
366
367      // ■■■■■■■■■■
368      public List<String> getLinkTags(long linkId) {
369          List<String> tags = new ArrayList<>();
370          String query = "SELECT " + LinkDbHelper.COLUMN_TAG_NAME +
371                  " FROM " + LinkDbHelper.TABLE_TAGS +
372                  " JOIN " + LinkDbHelper.TABLE_LINK_TAGS +
373                  " ON " + LinkDbHelper.TABLE_TAGS + "." + LinkDbHelper.COLUMN_TAG
374                  " = " + LinkDbHelper.TABLE_LINK_TAGS + "." + LinkDbHelper.COLUMN
375                  " WHERE " + LinkDbHelper.COLUMN_LINK_ID + " = ?";
376
377          Cursor cursor = database.rawQuery(query, new String[]{String.valueOf(lin
```

```
378          if (cursor.moveToFirst()) {
379              do {
380                  tags.add(cursor.getString(0));
381              } while (cursor.moveToNext());
382          }
383          cursor.close();
384          return tags;
385      }
386
387      // ■■■■■■■
388      public List<String> getAllTags() {
389          List<String> tags = new ArrayList<>();
390          Log.d("LinkDao", "Getting all tags");
391          Cursor cursor = database.query(
392                  LinkDbHelper.TABLE_TAGS,
393                  new String[]{LinkDbHelper.COLUMN_TAG_NAME},
394                  null, null, null, null, null);
395
396          Log.d("LinkDao", "Cursor count: " + cursor.getCount());
397          if (cursor.moveToFirst()) {
398              do {
399                  String tag = cursor.getString(0);
400                  Log.d("LinkDao", "Found tag: " + tag);
401                  tags.add(tag);
402              } while (cursor.moveToNext());
403          }
404          cursor.close();
405          return tags;
406      }
407
408      public void updateLinkTags(LinkItem item) {
409          // ■■■■■■■■■■■■
410          database.delete(
411                  LinkDbHelper.TABLE_LINK_TAGS,
412                  LinkDbHelper.COLUMN_LINK_ID + " = ?",
413                  new String[]{String.valueOf(item.getId())}
414          );
415
416          // ■■■■■■■■■
417          for (String tagName : item.getTags()) {
418              // ■■■■■■■■
419              long tagId = getOrCreateTag(tagName);
420              // ■■■■-■■■■
421              addTagToLink(item.getId(), tagId);
422          }
423      }
424
425      private long getOrCreateTag(String tagName) {
426          // ■■■■■■■■■
427          Cursor cursor = database.query(
```

```
428                 LinkDbHelper.TABLE_TAGS,
429                 new String[]{LinkDbHelper.COLUMN_TAG_ID},
430                 LinkDbHelper.COLUMN_TAG_NAME + " = ?",
431                 new String[]{tagName},
432                 null, null, null
433         );
434 
435         if (cursor.moveToFirst()) {
436             long tagId = cursor.getLong(0);
437             cursor.close();
438             return tagId;
439         }
440 
441         // ■■■■■■■■■■■■■■
442         cursor.close();
443         return addTag(tagName);
444     }
445 
446     // ■■■■
447     public long addTag(String tagName) {
448         ContentValues values = new ContentValues();
449         values.put(LinkDbHelper.COLUMN_TAG_NAME, tagName);
450         return database.insert(LinkDbHelper.TABLE_TAGS, null, values);
451     }
452 
453 
454     /**
455      * ■■■■■■■■■■■
456      * @param tag ■■■■■■■■
457      */
458     public void deleteTag(String tag) {
459         SQLiteDatabase db = dbHelper.getWritableDatabase();
460         db.beginTransaction();
461 
462         try {
463             // 1. ■■■■■■ID
464             long tagId = -1;
465             Cursor cursor = db.query(
466                     LinkDbHelper.TABLE_TAGS,
467                     new String[]{LinkDbHelper.COLUMN_TAG_ID},
468                     LinkDbHelper.COLUMN_TAG_NAME + " = ?",
469                     new String[]{tag},
470                     null, null, null
471             );
472 
473             if (cursor.moveToFirst()) {
474                 tagId = cursor.getLong(0);
475             }
476             cursor.close();
477 
```

```
478             if (tagId != -1) {
479                 // 2. ■■■■-■■■■■■■■■■
480                 Log.d("LinkDao", "■■■■■■■: tagId=" + tagId);
481                 int linkTagsDeleted = db.delete(
482                         LinkDbHelper.TABLE_LINK_TAGS,
483                         LinkDbHelper.COLUMN_TAG_ID_REF + " = ?",
484                         new String[]{String.valueOf(tagId)}
485                 );
486                 Log.d("LinkDao", "■■■" + linkTagsDeleted + "■■■■■■");
487 
488                 // 3. ■■■■■■■■■
489                 int tagsDeleted = db.delete(
490                         LinkDbHelper.TABLE_TAGS,
491                         LinkDbHelper.COLUMN_TAG_ID + " = ?",
492                         new String[]{String.valueOf(tagId)}
493                 );
494                 Log.d("LinkDao", "■■■" + tagsDeleted + "■■■");
495             } else {
496                 Log.w("LinkDao", "■■■■■: " + tag);
497             }
498 
499             db.setTransactionSuccessful();
500         } catch (Exception e) {
501             Log.e("LinkDao", "■■■■■■■■: " + e.getMessage(), e);
502         } finally {
503             db.endTransaction();
504         }
505     }
506 
507     /**
508      * ■■■■■■■■■■,■■■■■■■■■■■■■■
509      * @param tag ■■■■■■■■
510      */
511     public void deleteTagWithLinks(String tag) {
512         SQLiteDatabase db = dbHelper.getWritableDatabase();
513         db.beginTransaction();
514 
515         try {
516             // 1. ■■■■■■ID
517             long tagId = -1;
518             Cursor cursor = db.query(
519                     LinkDbHelper.TABLE_TAGS,
520                     new String[]{LinkDbHelper.COLUMN_TAG_ID},
521                     LinkDbHelper.COLUMN_TAG_NAME + " = ?",
522                     new String[]{tag},
523                     null, null, null
524             );
525 
526             if (cursor.moveToFirst()) {
527                 tagId = cursor.getLong(0);
```

```
528             }
529             cursor.close();
530
531             if (tagId != -1) {
532                 // 2. ■■■■■■■■■■■■■
533                 // ■■■■■■■■■■■■■■■ID
534                 String findLinksQuery = "SELECT " + LinkDbHelper.COLUMN_LINK_ID
535                                         " FROM " + LinkDbHelper.TABLE_LINK_TAGS +
536                                         " WHERE " + LinkDbHelper.COLUMN_TAG_ID_RE
537
538                 Cursor linksCursor = db.rawQuery(findLinksQuery, new String[]{St
539
540                 // ■■■■■■■■■ID■■
541                 List<Long> linksToDelete = new ArrayList<>();
542
543                 while (linksCursor.moveToNext()) {
544                     long linkId = linksCursor.getLong(0);
545
546                     // ■■■■■■■■■■■■■■■■■
547                     String countTagsQuery = "SELECT COUNT(*) FROM " + LinkDbHelp
548                                             " WHERE " + LinkDbHelper.COLUMN_LINK_
549
550                     Cursor tagCountCursor = db.rawQuery(countTagsQuery, new Stri
551
552                     if (tagCountCursor.moveToFirst() && tagCountCursor.getInt(0)
553                         // ■■■■■■■■■■■■■■■■■■■
554                         linksToDelete.add(linkId);
555                     }
556
557                     tagCountCursor.close();
558                 }
559                 linksCursor.close();
560
561                 // 3. ■■■■■■■■■■■■
562                 for (long linkId : linksToDelete) {
563                     Log.d("LinkDao", "■■■■■■■■■■■: linkId=" + linkId);
564                     db.delete(
565                             LinkDbHelper.TABLE_LINKS,
566                             LinkDbHelper.COLUMN_ID + " = ?",
567                             new String[]{String.valueOf(linkId)}
568                     );
569                 }
570
571                 // 4. ■■■■-■■■■
572                 int linkTagsDeleted = db.delete(
573                         LinkDbHelper.TABLE_LINK_TAGS,
574                         LinkDbHelper.COLUMN_TAG_ID_REF + " = ?",
575                         new String[]{String.valueOf(tagId)}
576                 );
577                 Log.d("LinkDao", "■■■" + linkTagsDeleted + "■■■■■");
```

```
578
579                 // 5. ■■■■■■
580                 int tagsDeleted = db.delete(
581                         LinkDbHelper.TABLE_TAGS,
582                         LinkDbHelper.COLUMN_TAG_ID + " = ?",
583                         new String[]{String.valueOf(tagId)}
584                 );
585                 Log.d("LinkDao", "■■■■■: " + tag);
586             } else {
587                 Log.w("LinkDao", "■■■■■: " + tag);
588             }
589
590             db.setTransactionSuccessful();
591         } catch (Exception e) {
592             Log.e("LinkDao", "■■■■■■■■■■: " + e.getMessage(), e);
593         } finally {
594             db.endTransaction();
595         }
596     }
597
598
599     // ■■■■■■■■ID■■■■
600     private void addTagToLink(long linkId, long tagId) {
601         ContentValues values = new ContentValues();
602         values.put(LinkDbHelper.COLUMN_LINK_ID, linkId);
603         values.put(LinkDbHelper.COLUMN_TAG_ID_REF, tagId);
604         database.insert(LinkDbHelper.TABLE_LINK_TAGS, null, values);
605     }
606
607     // ■■■■■■■■■■■■■■■■
608     public void addTagToLink(long linkId, String tagName) {
609         // ■■■■■■■■■■■■■■■ID
610         long tagId = getOrCreateTag(tagName);
611         // ■■■■-■■■■
612         addTagToLink(linkId, tagId);
613     }
614
615     public void ____________________________________(){}
616
617     // ■■■■■■■■■■ LinkItem ■■
618     private LinkItem createLinkItemFromCursor(Cursor cursor) {
619         long id = cursor.getLong(cursor.getColumnIndexOrThrow(LinkDbHelper.COLUM
620         String title = cursor.getString(cursor.getColumnIndexOrThrow(LinkDbHelpe
621         String url = cursor.getString(cursor.getColumnIndexOrThrow(LinkDbHelper.
622         String summary = cursor.getString(cursor.getColumnIndexOrThrow(LinkDbHel
623         String sourceApp = cursor.getString(cursor.getColumnIndexOrThrow(LinkDbH
624         String originalIntent = cursor.getString(cursor.getColumnIndexOrThrow(Li
625         String targetActivity = cursor.getString(cursor.getColumnIndexOrThrow(Li
626         long timestamp = cursor.getLong(cursor.getColumnIndexOrThrow(LinkDbHelpe
627         int clickCount = cursor.getInt(cursor.getColumnIndexOrThrow("click_count
```

```
628
629         // mark
630         LinkItem item = new LinkItem(title, url, sourceApp, originalIntent, targ
631         item.setId(id);
632         item.setSummary(summary);
633         item.setClickCount(clickCount);
634
635         // ■■■■■■■■
636         List<String> tags = getLinkTags(id);
637         for (String tag : tags) {
638             item.addTag(tag);
639         }
640
641         return item;
642     }
643
644     /**
645      * Get a single LinkItem by its id.
646      * Returns null if not found.
647      */
648     public LinkItem getLinkById(long linkId) {
649         Cursor cursor = database.query(
650                 LinkDbHelper.TABLE_LINKS,
651                 null,
652                 LinkDbHelper.COLUMN_ID + " = ?",
653                 new String[]{String.valueOf(linkId)},
654                 null, null, null, "1");
655         try {
656             if (cursor.moveToFirst()) {
657                 return createLinkItemFromCursor(cursor);
658             }
659             return null;
660         } finally {
661             cursor.close();
662         }
663     }
664
665     private LinkItem cursorToLinkItem(Cursor cursor) {
666         long id = cursor.getLong(cursor.getColumnIndexOrThrow(LinkDbHelper.COLUM
667         String title = cursor.getString(cursor.getColumnIndexOrThrow(LinkDbHelpe
668         String url = cursor.getString(cursor.getColumnIndexOrThrow(LinkDbHelper.
669         String summary = cursor.getString(cursor.getColumnIndexOrThrow(LinkDbHel
670         String sourceApp = cursor.getString(cursor.getColumnIndexOrThrow(LinkDbH
671         String originalIntent = cursor.getString(cursor.getColumnIndexOrThrow(Li
672         String targetActivity = cursor.getString(cursor.getColumnIndexOrThrow(Li
673         long timestamp = cursor.getLong(cursor.getColumnIndexOrThrow(LinkDbHelpe
674         boolean isPinned = cursor.getInt(cursor.getColumnIndexOrThrow("is_pinned
675         int clickCount = cursor.getInt(cursor.getColumnIndexOrThrow("click_count
676
677
```

```
678          LinkItem item = new LinkItem(title, url, sourceApp, originalIntent, targ
679          item.setId(id);
680          item.setSummary(summary);
681          item.setPinned(isPinned);
682          item.setClickCount(clickCount);
683          return item;
684      }
685
686      // ■■■■■■■■■■
687      private String makePlaceholders(int count) {
688          if (count < 1) return "";
689          StringBuilder sb = new StringBuilder(count * 2 - 1);
690          sb.append("?");
691          for (int i = 1; i < count; i++) {
692              sb.append(",?");
693          }
694          return sb.toString();
695      }
696
697
698
699      public Map<String, Integer> getDailyStatistics() {
700          Log.d(TAG, "getDailyStatistics: ■■■■■■■■■■");
701          Map<String, Integer> statistics = new HashMap<>();
702          Cursor cursor = null;
703          try {
704              String query = "SELECT date(timestamp/1000, 'unixepoch') as date, CO
705                             "FROM links GROUP BY date(timestamp/1000, 'unixepoch')
706              Log.d(TAG, "getDailyStatistics: SQL■■: " + query);
707              cursor = database.rawQuery(query, null);
708
709              Log.d(TAG, "getDailyStatistics: ■■■■■■: " + cursor.getCount());
710              while (cursor.moveToNext()) {
711                  String date = cursor.getString(0);
712                  int count = cursor.getInt(1);
713                  Log.d(TAG, "getDailyStatistics: ■■■■: " + date + " -> " + count)
714                  statistics.put(date, count);
715              }
716          } finally {
717              if (cursor != null) {
718                  cursor.close();
719              }
720          }
721          Log.d(TAG, "getDailyStatistics: ■■■■■■ " + statistics.size() + " ■■■");
722          return statistics;
723      }
724
725      public Cursor getClickStatistics(String period) {
726          String query = "SELECT strftime('%Y-%m', datetime(timestamp/1000, 'unixe
727                  "SUM(click_count) as total_clicks " +
```

```
728                 "FROM links " +
729                 "GROUP BY period " +
730                 "ORDER BY period DESC";
731
732         if ("week".equals(period)) {
733             query = "SELECT strftime('%Y-%W', datetime(timestamp/1000, 'unixepoc
734                     "SUM(click_count) as total_clicks " +
735                     "FROM links " +
736                     "GROUP BY period " +
737                     "ORDER BY period DESC";
738         }
739
740         return database.rawQuery(query, null);
741     }
742
743     /**
744      * ■■■■■■■■■■■■■■■■■■■■■■
745      * @return Map<String, Integer> ■■■■■■■■■■■■
746      */
747     public Map<String, Integer> getTagsWithCount() {
748         Map<String, Integer> tagCountMap = new LinkedHashMap<>();
749         SQLiteDatabase db = dbHelper.getReadableDatabase();
750
751         // 1. ■■■■■■■■
752         List<Long> orderedTagIds = getTagOrder();
753         Set<Long> orderedSet = new HashSet<>(orderedTagIds);
754
755         // 2. ■■■■■■■■■■■■■■■■
756         Map<Long, TagCountInfo> allTags = new HashMap<>();
757         Cursor cursor = db.rawQuery(
758             "SELECT t." + LinkDbHelper.COLUMN_TAG_ID +
759             ", t." + LinkDbHelper.COLUMN_TAG_NAME +
760             ", COUNT(lt." + LinkDbHelper.COLUMN_LINK_ID + ") as count " +
761             "FROM " + LinkDbHelper.TABLE_TAGS + " t " +
762             "LEFT JOIN " + LinkDbHelper.TABLE_LINK_TAGS + " lt " +
763             "ON t." + LinkDbHelper.COLUMN_TAG_ID + " = lt." + LinkDbHelper.COLUM
764             "GROUP BY t." + LinkDbHelper.COLUMN_TAG_ID + ", t." + LinkDbHelper.C
765
766         if (cursor.moveToFirst()) {
767             do {
768                 long tagId = cursor.getLong(0);
769                 String tagName = cursor.getString(1);
770                 int count = cursor.getInt(2);
771                 // ■■■■■■■■■■count■0■■■
772                 allTags.put(tagId, new TagCountInfo(tagName, count));
773             } while (cursor.moveToNext());
774         }
775         cursor.close();
776
777         // 3. ■■■■■■■■■
```

```
778          for (Long tagId : orderedTagIds) {
779              TagCountInfo info = allTags.get(tagId);
780              if (info != null) {
781                  tagCountMap.put(info.name, info.count);
782              }
783          }
784
785          // 4. ■■■■■■■■■■■■■■tag_id■■■
786          List<Long> unorderedTagIds = new ArrayList<>();
787          for (Long tagId : allTags.keySet()) {
788              if (!orderedSet.contains(tagId)) {
789                  unorderedTagIds.add(tagId);
790              }
791          }
792          Collections.sort(unorderedTagIds);
793          for (Long tagId : unorderedTagIds) {
794              TagCountInfo info = allTags.get(tagId);
795              if (info != null) {
796                  tagCountMap.put(info.name, info.count);
797              }
798          }
799
800          return tagCountMap;
801      }
802
803      // ■■■
804      private static class TagCountInfo {
805          String name;
806          int count;
807          TagCountInfo(String name, int count) {
808              this.name = name;
809              this.count = count;
810          }
811      }
812
813      public void updateLinkRemark(long linkId, String remark) {
814          ContentValues values = new ContentValues();
815          values.put(LinkDbHelper.COLUMN_REMARK, remark);
816
817          database.update(
818                  LinkDbHelper.TABLE_LINKS,
819                  values,
820                  LinkDbHelper.COLUMN_ID + " = ?",
821                  new String[]{String.valueOf(linkId)}
822          );
823
824          Log.d("LinkDao", "■■■■■■: linkId=" + linkId + ", remark is " + remark);
825      }
826
827      /**
```

```
828         * ■■■■■■■
829         * @param tagIds ■■■■■■■■■■■ID■■
830         */
831        public void saveTagOrder(List<Long> tagIds) {
832            try {
833                JSONArray jsonArray = new JSONArray();
834                for (Long tagId : tagIds) {
835                    jsonArray.put(tagId);
836                }
837
838                ContentValues values = new ContentValues();
839                values.put(LinkDbHelper.COLUMN_CONFIG_KEY, "tag_order");
840                values.put(LinkDbHelper.COLUMN_CONFIG_VALUE, jsonArray.toString());
841
842                // ■■■■■■■■■■■ INSERT OR REPLACE■
843                database.delete(LinkDbHelper.TABLE_CONFIG,
844                    LinkDbHelper.COLUMN_CONFIG_KEY + " = ?",
845                    new String[]{"tag_order"});
846                database.insert(LinkDbHelper.TABLE_CONFIG, null, values);
847
848                Log.d("LinkDao", "■■■■■■■: " + tagIds.size() + " ■■■■");
849            } catch (Exception e) {
850                Log.e("LinkDao", "■■■■■■■■■", e);
851            }
852        }
853
854        /**
855         * ■■■■■■■
856         * @return ■■■■■■■■■■■ID■■■■■■■■■■■■■■■
857         */
858        public List<Long> getTagOrder() {
859            List<Long> tagIds = new ArrayList<>();
860            Cursor cursor = database.query(
861                LinkDbHelper.TABLE_CONFIG,
862                new String[]{LinkDbHelper.COLUMN_CONFIG_VALUE},
863                LinkDbHelper.COLUMN_CONFIG_KEY + " = ?",
864                new String[]{"tag_order"},
865                null, null, null);
866
867            if (cursor.moveToFirst()) {
868                try {
869                    String value = cursor.getString(0);
870                    JSONArray jsonArray = new JSONArray(value);
871                    for (int i = 0; i < jsonArray.length(); i++) {
872                        tagIds.add(jsonArray.getLong(i));
873                    }
874                    Log.d("LinkDao", "■■■■■■■: " + tagIds.size() + " ■■■■");
875                } catch (JSONException e) {
876                    Log.e("LinkDao", "■■■■■■■■■", e);
877                }
```

```
878          }
879          cursor.close();
880          return tagIds;
881      }
882
883      /**
884       * ■■■■ID■■■■■■
885       */
886      public String getTagNameById(long tagId) {
887          Cursor cursor = database.query(
888              LinkDbHelper.TABLE_TAGS,
889              new String[]{LinkDbHelper.COLUMN_TAG_NAME},
890              LinkDbHelper.COLUMN_TAG_ID + " = ?",
891              new String[]{String.valueOf(tagId)},
892              null, null, null);
893
894          String tagName = null;
895          if (cursor.moveToFirst()) {
896              tagName = cursor.getString(0);
897          }
898          cursor.close();
899          return tagName;
900      }
901
902      /**
903       * ■■■■■■■■■■ID
904       */
905      public long getTagIdByName(String tagName) {
906          Cursor cursor = database.query(
907              LinkDbHelper.TABLE_TAGS,
908              new String[]{LinkDbHelper.COLUMN_TAG_ID},
909              LinkDbHelper.COLUMN_TAG_NAME + " = ?",
910              new String[]{tagName},
911              null, null, null);
912
913          long tagId = -1;
914          if (cursor.moveToFirst()) {
915              tagId = cursor.getLong(0);
916          }
917          cursor.close();
918          return tagId;
919      }
920  }
  1  // MISSING: person/notfresh/readingshare/db/DatabaseHelper.java
  1  // MISSING: person/notfresh/readingshare/model/Link.java
  1  // MISSING: person/notfresh/readingshare/model/Tag.java
  1  // MISSING: person/notfresh/readingshare/ui/tag/TagFragment.java
  1  // MISSING: person/notfresh/readingshare/adapter/TagAdapter.java
  1  package person.notfresh.readingshare.ui.subject;
  2
```

```
 3  import android.content.Intent;
 4  import android.os.Bundle;
 5  import android.util.Log;
 6  import android.view.Menu;
 7  import android.view.MenuItem;
 8  import android.widget.Toast;
 9
10  import androidx.annotation.NonNull;
11  import androidx.appcompat.app.AppCompatActivity;
12  import androidx.appcompat.widget.Toolbar;
13  import androidx.recyclerview.widget.ItemTouchHelper;
14  import androidx.recyclerview.widget.LinearLayoutManager;
15  import androidx.recyclerview.widget.RecyclerView;
16
17  import java.util.List;
18
19  import person.notfresh.readingshare.R;
20  import person.notfresh.readingshare.WebViewActivity;
21  import person.notfresh.readingshare.adapter.SubjectItemAdapter;
22  import person.notfresh.readingshare.core.model.Subject;
23  import person.notfresh.readingshare.core.model.SubjectItem;
24  import person.notfresh.readingshare.core.model.SubjectUtil;
25  import person.notfresh.readingshare.db.LinkDao;
26  import person.notfresh.readingshare.db.SubjectDao;
27  import person.notfresh.readingshare.model.LinkItem;
28  import person.notfresh.readingshare.ui.subject.AddSubjectItemDialog;
29  import person.notfresh.readingshare.ui.subject.SubjectArchivedItemsDialog;
30
31  /**
32   * ■■■■■Activity
33   */
34  public class SubjectDetailActivity extends AppCompatActivity implements
35          SubjectItemAdapter.OnSubjectItemClickListener,
36          SubjectItemAdapter.OnSubjectItemActionListener {
37      private static final String TAG = "SubjectDetailActivity";
38      public static final String EXTRA_SUBJECT_ID = "subject_id";
39
40      private RecyclerView recyclerView;
41      private android.widget.TextView textEmpty;
42      private SubjectItemAdapter adapter;
43      private SubjectDao subjectDao;
44      private LinkDao linkDao;
45      private Subject subject;
46      private long subjectId;
47      private ItemTouchHelper itemTouchHelper;
48      private List<SubjectItem> currentSubjectItems; // ■■■■■■■■■■■■■■ context_ids
49
50      @Override
51      protected void onCreate(Bundle savedInstanceState) {
52          super.onCreate(savedInstanceState);
```

```
53          setContentView(R.layout.activity_subject_detail);
54
55          // ■■■■ID
56          subjectId = getIntent().getLongExtra(EXTRA_SUBJECT_ID, -1);
57          if (subjectId == -1) {
58              Toast.makeText(this, "■■ID■■", Toast.LENGTH_SHORT).show();
59              finish();
60              return;
61          }
62
63          // ■■Toolbar
64          Toolbar toolbar = findViewById(R.id.toolbar);
65          setSupportActionBar(toolbar);
66          if (getSupportActionBar() != null) {
67              getSupportActionBar().setDisplayHomeAsUpEnabled(true);
68          }
69
70          subjectDao = new SubjectDao(this);
71          subjectDao.open();
72          linkDao = new LinkDao(this);
73          linkDao.open();
74
75          // ■■■■
76          subject = subjectDao.getSubjectById(subjectId);
77          if (subject == null) {
78              Toast.makeText(this, "■■■■■", Toast.LENGTH_SHORT).show();
79              finish();
80              return;
81          }
82
83          // ■■■■
84          if (getSupportActionBar() != null) {
85              getSupportActionBar().setTitle(subject.getTitle());
86          }
87
88          recyclerView = findViewById(R.id.recycler_view);
89          textEmpty = findViewById(R.id.text_empty);
90          adapter = new SubjectItemAdapter(this);
91          adapter.setOnSubjectItemClickListener(this);
92          adapter.setOnSubjectItemActionListener(this);
93          recyclerView.setAdapter(adapter);
94          recyclerView.setLayoutManager(new LinearLayoutManager(this));
95
96          // ■■■■■■
97          setupDragAndDrop();
98
99          loadSubjectItems();
100     }
101
102     @Override
```

```
103      protected void onDestroy() {
104          super.onDestroy();
105          if (adapter != null) {
106              adapter.close();
107          }
108          if (subjectDao != null) {
109              subjectDao.close();
110          }
111          if (linkDao != null) {
112              linkDao.close();
113          }
114      }
115
116      @Override
117      public boolean onCreateOptionsMenu(Menu menu) {
118          getMenuInflater().inflate(R.menu.subject_detail_menu, menu);
119          return true;
120      }
121
122      @Override
123      public boolean onOptionsItemSelected(@NonNull MenuItem item) {
124          if (item.getItemId() == android.R.id.home) {
125              finish();
126              return true;
127          } else if (item.getItemId() == R.id.action_add_item) {
128              showAddSubjectItemDialog();
129              return true;
130          } else if (item.getItemId() == R.id.action_subject_settings) {
131              showSubjectSettingsDialog();
132              return true;
133          }
134          return super.onOptionsItemSelected(item);
135      }
136
137      private void showAddSubjectItemDialog() {
138          AddSubjectItemDialog dialog = AddSubjectItemDialog.newInstance(subjectId
139          dialog.setOnSubjectItemSavedListener(item -> {
140              // ■■■■
141              loadSubjectItems();
142              Toast.makeText(this, "■■■■■■■■", Toast.LENGTH_SHORT).show();
143          });
144          dialog.show(getSupportFragmentManager(), "AddSubjectItemDialog");
145      }
146
147      private void loadSubjectItems() {
148          // ■■■■■■■■■■■■■■■■■■
149          subject = subjectDao.getSubjectById(subjectId);
150          if (subject != null) {
151              currentSubjectItems = subject.getSubItems();
152              adapter.setItems(currentSubjectItems);
```

```
153             updateEmptyState(currentSubjectItems);
154             Log.d(TAG, "■■■ " + currentSubjectItems.size() + " ■■■■");
155         } else {
156             currentSubjectItems = null;
157             updateEmptyState(null);
158         }
159     }
160 
161     private void updateEmptyState(List<SubjectItem> items) {
162         if (textEmpty == null) {
163             return;
164         }
165         boolean isEmpty = items == null || items.isEmpty();
166         textEmpty.setVisibility(isEmpty ? android.view.View.VISIBLE : android.vi
167         recyclerView.setVisibility(isEmpty ? android.view.View.GONE : android.vi
168     }
169 
170     @Override
171     public void onSubjectItemClick(SubjectItem item) {
172         // ■■■■■ID■■■ context_ids ■■■ WebViewActivity
173         if (item.getLinkId() != null && item.getLinkId() > 0) {
174             openWebViewWithContext(item);
175         } else {
176             // ■■■■■■■■■■■■
177             showEditSubjectItemDialog(item);
178         }
179     }
180 
181     /**
182      * ■■ WebViewActivity ■■■■■■■■
183      */
184     private void openWebViewWithContext(SubjectItem clickedItem) {
185         if (currentSubjectItems == null || currentSubjectItems.isEmpty()) {
186             return;
187         }
188 
189         // ■■■■■■■■ SubjectItem ■ linkId
190         java.util.ArrayList<Long> contextIdsList = new java.util.ArrayList<>();
191         int clickedIndex = -1;
192 
193         for (int i = 0; i < currentSubjectItems.size(); i++) {
194             SubjectItem item = currentSubjectItems.get(i);
195             if (item.getLinkId() != null && item.getLinkId() > 0) {
196                 contextIdsList.add(item.getLinkId());
197                 if (clickedIndex == -1 && item.getId() == clickedItem.getId()) {
198                     clickedIndex = contextIdsList.size() - 1;
199                 }
200             }
201         }
202 
```

```
203          if (contextIdsList.size() <= 1 || clickedIndex == -1) {
204              // ■■■■■■■■■■■■■■■■■■■■■
205              showEditSubjectItemDialog(clickedItem);
206              return;
207          }
208
209          // ■■■■■ URL
210          LinkItem linkItem = linkDao.getLinkById(clickedItem.getLinkId());
211          if (linkItem == null || linkItem.getUrl() == null) {
212              Toast.makeText(this, "■■■■■■■", Toast.LENGTH_SHORT).show();
213              return;
214          }
215
216          // ■■ context_ids ■■
217          long[] contextIds = new long[contextIdsList.size()];
218          for (int i = 0; i < contextIdsList.size(); i++) {
219              contextIds[i] = contextIdsList.get(i);
220          }
221
222          // ■■ WebViewActivity
223          Intent intent = new Intent(this, WebViewActivity.class);
224          intent.putExtra("url", linkItem.getUrl());
225          intent.putExtra("context_ids", contextIds);
226          intent.putExtra("context_index", clickedIndex);
227          startActivity(intent);
228      }
229
230      private void showEditSubjectItemDialog(SubjectItem item) {
231          AddSubjectItemDialog dialog = AddSubjectItemDialog.newInstance(subjectId
232          dialog.setOnSubjectItemSavedListener(editedItem -> {
233              // ■■■■
234              loadSubjectItems();
235              Toast.makeText(this, "■■■■■■■■", Toast.LENGTH_SHORT).show();
236          });
237          dialog.show(getSupportFragmentManager(), "EditSubjectItemDialog");
238      }
239
240      /**
241       * ■■■■■■■
242       */
243      private void setupDragAndDrop() {
244          ItemTouchHelper.Callback callback = new ItemTouchHelper.SimpleCallback(
245                  ItemTouchHelper.UP | ItemTouchHelper.DOWN, 0) {
246              @Override
247              public boolean onMove(@NonNull RecyclerView recyclerView,
248                                    @NonNull RecyclerView.ViewHolder viewHolder,
249                                    @NonNull RecyclerView.ViewHolder target) {
250                  int fromPos = viewHolder.getAdapterPosition();
251                  int toPos = target.getAdapterPosition();
252
```

```
253                 // ■■■■■■■
254                 List<SubjectItem> items = adapter.getItems();
255                 if (fromPos < 0 || fromPos >= items.size() || toPos < 0 || toPos
256                     return false;
257                 }
258
259                 // ■■■■■
260                 SubjectItem draggedItem = items.get(fromPos);
261                 items.remove(fromPos);
262                 items.add(toPos, draggedItem);
263                 adapter.notifyItemMoved(fromPos, toPos);
264
265                 return true;
266             }
267
268             @Override
269             public void onSwiped(@NonNull RecyclerView.ViewHolder viewHolder, in
270                 // ■■■■■■
271             }
272
273             @Override
274             public void clearView(@NonNull RecyclerView recyclerView,
275                                   @NonNull RecyclerView.ViewHolder viewHolder) {
276                 super.clearView(recyclerView, viewHolder);
277
278                 // ■■■■■■■■■■■■■■■■■■■ orderIndex ■■■■
279                 // ■■■■■■■■■■■■■ onMove ■■■■■■■■■■■■■■■■■■■■■■■
280                 List<SubjectItem> items = adapter.getItems();
281
282                 // ■■■■■■■■■■■■■■■■■■■■■■■■■■ orderIndex
283                 for (int i = 0; i < items.size(); i++) {
284                     items.get(i).setOrderIndex(i * SubjectUtil.ORDER_INTERVAL);
285                 }
286
287                 // ■■■■■■■
288                 // ■■■updateSubjectItem ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
289                 int successCount = 0;
290                 for (SubjectItem item : items) {
291                     if (subjectDao.updateSubjectItem(item)) {
292                         successCount++;
293                     }
294                 }
295
296                 Log.d(TAG, "■■■■■■■■■■■ " + successCount + "/" + items.size() + "
297
298                 // ■■■■■■■■■■■■■■■■■■
299                 // loadSubjectItems(); // ■■■■■■■■■■■■■■■■■■■■
300             }
301         };
302
```

```
303          itemTouchHelper = new ItemTouchHelper(callback);
304          itemTouchHelper.attachToRecyclerView(recyclerView);
305      }
306
307      @Override
308      public void onCollectLink(SubjectItem item, LinkItem linkItem) {
309          // ■■■■■■■■■■■■■■
310          List<LinkItem> existingLinks = linkDao.getAllLinks();
311          boolean linkExists = false;
312          for (LinkItem existing : existingLinks) {
313              if (existing.getUrl() != null && existing.getUrl().equals(linkItem.g
314                  linkExists = true;
315                  break;
316              }
317          }
318
319          if (!linkExists) {
320              // ■■■■■■■■■■■■
321              linkDao.insertLink(linkItem);
322              Toast.makeText(this, "■■■■■■■■■", Toast.LENGTH_SHORT).show();
323          } else {
324              Toast.makeText(this, "■■■■■■■■■", Toast.LENGTH_SHORT).show();
325          }
326      }
327
328      @Override
329      public void onDeleteSubjectItem(SubjectItem item) {
330          // ■■■■■■■■■■■■■■■
331          boolean deleted = subjectDao.deleteSubjectItem(item.getId());
332          if (deleted) {
333              loadSubjectItems();
334              Toast.makeText(this, "■■■■■■■", Toast.LENGTH_SHORT).show();
335          } else {
336              Toast.makeText(this, "■■■■■", Toast.LENGTH_SHORT).show();
337          }
338      }
339
340      @Override
341      public void onArchiveSubjectItem(SubjectItem item) {
342          long archivedAt = System.currentTimeMillis();
343          boolean archived = subjectDao.archiveSubjectItem(item.getId(), archivedA
344          if (archived) {
345              item.setArchived(true);
346              item.setArchivedAt(archivedAt);
347              if (adapter != null) {
348                  adapter.removeItem(item);
349              }
350              Toast.makeText(this, "■■■■■■■■■-■■■■■■■", Toast.LENGTH_SHORT).show();
351          } else {
352              Toast.makeText(this, "■■■■■", Toast.LENGTH_SHORT).show();
```

```
353         }
354     }
355
356     @Override
357     public void onRestoreSubjectItem(SubjectItem item) {
358         boolean restored = subjectDao.restoreSubjectItem(item.getId());
359         if (restored) {
360             item.setArchived(false);
361             item.setArchivedAt(0);
362             loadSubjectItems();
363             Toast.makeText(this, "■■■", Toast.LENGTH_SHORT).show();
364         } else {
365             Toast.makeText(this, "■■■■", Toast.LENGTH_SHORT).show();
366         }
367     }
368
369     @Override
370     public void onRefreshItems() {
371         loadSubjectItems();
372     }
373
374     private void showSubjectSettingsDialog() {
375         android.view.View view = getLayoutInflater().inflate(R.layout.dialog_sub
376         android.widget.Button btnArchived = view.findViewById(R.id.btn_archived)
377
378         androidx.appcompat.app.AlertDialog dialog = new androidx.appcompat.app.A
379                 .setTitle("■■")
380                 .setView(view)
381                 .setNegativeButton("■■", null)
382                 .create();
383
384         btnArchived.setOnClickListener(v -> {
385             dialog.dismiss();
386             showArchivedItemsDialog();
387         });
388
389         dialog.show();
390     }
391
392     private void showArchivedItemsDialog() {
393         SubjectArchivedItemsDialog dialog = SubjectArchivedItemsDialog.newInstan
394         dialog.setOnItemsRestoredListener(() -> loadSubjectItems());
395         dialog.show(getSupportFragmentManager(), "SubjectArchivedItemsDialog");
396     }
397 }
398
```

```
1  package person.notfresh.readingshare.ui.archive;
2
3  import android.content.ClipData;
4  import android.content.ComponentName;
```

```
 5  import android.content.ContentResolver;
 6  import android.content.Intent;
 7  import android.database.Cursor;
 8  import android.net.Uri;
 9  import android.os.Build;
10  import android.os.Bundle;
11  import android.provider.MediaStore;
12  import android.text.Editable;
13  import android.text.TextWatcher;
14  import android.util.Log;
15  import android.view.LayoutInflater;
16  import android.view.Menu;
17  import android.view.MenuInflater;
18  import android.view.MenuItem;
19  import android.view.View;
20  import android.view.ViewGroup;
21  import android.widget.EditText;
22  import android.widget.Toast;
23
24  import androidx.annotation.NonNull;
25  import androidx.annotation.Nullable;
26  import androidx.fragment.app.Fragment;
27  import androidx.navigation.Navigation;
28  import androidx.recyclerview.widget.LinearLayoutManager;
29  import androidx.recyclerview.widget.RecyclerView;
30
31  import com.google.android.material.snackbar.Snackbar;
32
33  import java.io.File;
34  import java.util.ArrayList;
35  import java.util.List;
36  import java.util.Map;
37  import java.util.Set;
38
39  import person.notfresh.readingshare.ClickStatisticsActivity;
40  import person.notfresh.readingshare.R;
41  import person.notfresh.readingshare.adapter.LinksAdapter;
42  import person.notfresh.readingshare.databinding.FragmentHomeBinding;
43  import person.notfresh.readingshare.db.LinkDao;
44  import person.notfresh.readingshare.model.LinkItem;
45  import person.notfresh.readingshare.util.ExportUtil;
46  import person.notfresh.readingshare.util.ShareUtil;
47
48  public class ArchiveFragment extends Fragment implements LinksAdapter.OnLinkActi
49
50      private FragmentHomeBinding binding;
51      private LinksAdapter adapter;
52      private LinkDao linkDao;
53      private boolean isSelectionMode = false;
54      private MenuItem shareMenuItem;
```

```
55      private MenuItem closeSelectionMenuItem;
56      private EditText searchEditText;
57
58      @Override
59      public void onCreate(@Nullable Bundle savedInstanceState) {
60          super.onCreate(savedInstanceState);
61          setHasOptionsMenu(true);
62          Log.d("HomeFragment", "onCreate: setHasOptionsMenu(true)");
63      }
64
65      @Override
66      public void onCreateOptionsMenu(@NonNull Menu menu, @NonNull MenuInflater in
67          Log.d("HomeFragment", "onCreateOptionsMenu");
68          menu.clear();
69          inflater.inflate(R.menu.home_menu, menu);
70          shareMenuItem = menu.findItem(R.id.action_share);
71          closeSelectionMenuItem = menu.findItem(R.id.action_close_selection);
72          MenuItem statisticsMenuItem = menu.findItem(R.id.action_statistics);  //
73
74          // ■■■■■■■■■■■■
75          View actionView = requireActivity().findViewById(statisticsMenuItem.getI
76          if (actionView != null) {
77              ViewGroup.MarginLayoutParams params = (ViewGroup.MarginLayoutParams)
78              params.rightMargin = getResources().getDimensionPixelSize(R.dimen.st
79              actionView.setLayoutParams(params);
80          }
81
82         //■■■■■■■■■■■
83      //     MenuItem clickStatsButton = menu.findItem(R.id.action_statistics_read
84      //    clickStatsButton.setOnClickListener(v -> {
85      //        // ■■■■■■■■■■
86      //        Intent intent = new Intent(getActivity(), ClickStatisticsActivity.
87      //        startActivity(intent);
88      //    });
89
90          shareMenuItem.setVisible(isSelectionMode);
91          closeSelectionMenuItem.setVisible(isSelectionMode);
92
93          super.onCreateOptionsMenu(menu, inflater);
94      }
95
96      @Override
97      public boolean onOptionsItemSelected(@NonNull MenuItem item) {
98          int id = item.getItemId();
99          if (id == R.id.action_close_selection) {
100             toggleSelectionMode();  // ■■■■■■■
101             return true;
102         } else if (id == R.id.action_share_text) {
103             shareAsText();
104             return true;
```

```
105          } else if (id == R.id.action_share_json) {
106              shareAsFile(true);  // true ■■ JSON
107              return true;
108          } else if (id == R.id.action_share_csv) {
109              shareAsFile(false);  // false ■■ CSV
110              return true;
111          } else if (id == R.id.action_statistics) {
112              // ■■■■■■■■
113              Navigation.findNavController(requireView())
114                      .navigate(R.id.action_nav_home_to_nav_statistics);
115              return true;
116          } else if (id == R.id.action_statistics_read) {
117              // ■■■■■■■■■■
118              Intent intent = new Intent(getActivity(), ClickStatisticsActivity.cl
119              startActivity(intent);
120              return true;
121          }
122          return super.onOptionsItemSelected(item);
123      }
124
125
126      @Override
127      public View onCreateView(@NonNull LayoutInflater inflater,
128                               ViewGroup container, Bundle savedInstanceState) {
129          binding = FragmentHomeBinding.inflate(inflater, container, false);
130          View root = binding.getRoot();
131
132          // ■■■■■■■■■■
133          String selectedDate = null;
134          if (getArguments() != null) {
135              selectedDate = getArguments().getString("selected_date");
136          }
137          String archive_db = "archive.db";
138          linkDao = new LinkDao(requireContext(), archive_db);
139          linkDao.open();
140
141          RecyclerView recyclerView = binding.recyclerView;
142          adapter = new LinksAdapter(requireContext(), archive_db, "archive");
143          adapter.setOnLinkActionListener(this);
144          recyclerView.setAdapter(adapter);
145          recyclerView.setLayoutManager(new LinearLayoutManager(requireContext()))
146
147          // ■■■■■■■■
148          // adapter.enableSwipeActions(recyclerView);
149
150          // ■■ RecyclerView ■■■■■
151          recyclerView.setOnTouchListener((v, event) -> {
152              searchEditText.clearFocus();  // ■■■■■■■■
153              return false;
154          });
```

```
155
156         // ■■■■■■■■■■■■
157         List<LinkItem> pinnedLinks = linkDao.getPinnedLinks(); // ■■■ LinkDao ■■
158         Map<String, List<LinkItem>> groupedLinks = linkDao.getLinksGroupByDate()
159
160         // ■■■■■■■■■■■■■■■
161         adapter.setPinnedLinks(pinnedLinks); // ■■■ LinksAdapter ■■■■■■
162         adapter.setGroupedLinks(groupedLinks);
163
164         // ■■■■■■■■■■■■■■■■
165         if (selectedDate != null) {
166             scrollToDate(recyclerView, selectedDate);
167         }
168
169         // ■■■■■
170         searchEditText = binding.searchEditText;
171         searchEditText.addTextChangedListener(new TextWatcher() {
172             @Override
173             public void beforeTextChanged(CharSequence s, int start, int count,
174
175             @Override
176             public void onTextChanged(CharSequence s, int start, int before, int
177
178             @Override
179             public void afterTextChanged(Editable s) {
180                 adapter.filter(s.toString());
181             }
182         });
183
184         return root;
185     }
186
187     @Override
188     public void onDestroyView() {
189         super.onDestroyView();
190         binding = null;
191         if (linkDao != null) {
192             linkDao.close();
193         }
194     }
195
196
197     @Override
198     public void onDeleteLink(LinkItem link) {
199         // linkDao.deleteLink(link.getUrl());
200         linkDao.deleteLink(link.getId());
201         // ■■■■
202         Map<String, List<LinkItem>> groupedLinks = linkDao.getLinksGroupByDate()
203         adapter.setGroupedLinks(groupedLinks);
204     }
```

```
205
206      @Override
207      public void onUpdateLink(LinkItem oldLink, String newTitle) {
208          linkDao.updateLinkTitle(oldLink.getUrl(), newTitle);
209          // ■■■■
210          Map<String, List<LinkItem>> groupedLinks = linkDao.getLinksGroupByDate()
211          adapter.setGroupedLinks(groupedLinks);
212      }
213
214      @Override
215      public boolean deleteLink(Long id) {
216          return false;
217      }
218
219      //@Override
220      public void addTagToLink(LinkItem item, String tag) {
221          linkDao.addTagToLink(item.getId(), tag);
222          // ■■■■
223          Map<String, List<LinkItem>> groupedLinks = linkDao.getLinksGroupByDate()
224          adapter.setGroupedLinks(groupedLinks);
225      }
226
227
228
229      @Override
230      public void addTagsToLink(LinkItem item, List<String> tags) {
231
232      }
233
234      //@Override
235      public void updateLinkTags(LinkItem item) {
236          linkDao.updateLinkTags(item);
237          // ■■■■
238          Map<String, List<LinkItem>> groupedLinks = linkDao.getLinksGroupByDate()
239          adapter.setGroupedLinks(groupedLinks);
240      }
241
242      @Override
243      public void onEnterSelectionMode() {
244          if (!isSelectionMode) {
245              toggleSelectionMode();
246          }
247      }
248
249      private void toggleSelectionMode() {
250          Log.d("HomeFragment", "toggleSelectionMode called");
251          isSelectionMode = !isSelectionMode;
252          adapter.toggleSelectionMode();
253          if (shareMenuItem != null) {
254              shareMenuItem.setVisible(isSelectionMode);
```

```
255          }
256          if (closeSelectionMenuItem != null) {
257              closeSelectionMenuItem.setVisible(isSelectionMode);
258          }
259          // ■■■■
260          if (isSelectionMode) {
261              requireActivity().setTitle("■■■■■■■■■");
262          } else {
263              requireActivity().setTitle(R.string.app_name);
264          }
265          requireActivity().invalidateOptionsMenu();
266          Log.d("HomeFragment", "Selection mode: " + isSelectionMode);
267      }
268
269      private void shareAsText() {
270          Set<LinkItem> selectedItems = adapter.getSelectedItems();
271          if (selectedItems.isEmpty()) {
272              Toast.makeText(requireContext(), "■■■■■■■■■■■", Toast.LENGTH_SHORT).s
273              return;
274          }
275
276          StringBuilder shareText = new StringBuilder();
277          for (LinkItem item : selectedItems) {
278              shareText.append(item.getTitle())
279                      .append("\n")
280                      .append(item.getUrl())
281                      .append("\n\n");
282          }
283
284          Intent shareIntent = new Intent(Intent.ACTION_SEND);
285          shareIntent.setType("text/plain");
286          shareIntent.putExtra(Intent.EXTRA_TEXT, shareText.toString());
287
288          // ■■■■■■■■■■■■■■
289          Intent chooserIntent = Intent.createChooser(shareIntent, "■■■■");
290          String myPackageName = requireContext().getPackageName();
291          chooserIntent.addFlags(Intent.FLAG_ACTIVITY_EXCLUDE_FROM_RECENTS);
292          chooserIntent.putExtra(Intent.EXTRA_EXCLUDE_COMPONENTS,
293              new ComponentName[]{new ComponentName(myPackageName, myPackageName +
294
295          startActivity(chooserIntent);
296      }
297
298      private void shareAsFile(boolean isJson) {
299          Set<LinkItem> selectedItems = adapter.getSelectedItems();
300          if (selectedItems.isEmpty()) {
301              Toast.makeText(requireContext(), "■■■■■■■■■■■", Toast.LENGTH_SHORT).s
302              return;
303          }
304
```

```
305          try {
306              // ■■■■■■ Documents ■■
307              Uri fileUri = ExportUtil.exportToPublicDirectory(
308                  requireContext(),
309                  new ArrayList<>(selectedItems),
310                  isJson
311              );
312
313              // ■■■■■■■■■■■■■
314              String fileName = isJson
315                  ? "links_" + ExportUtil.getCurrentTime() + "_readshare.json"
316                  : "links_" + ExportUtil.getCurrentTime() + "_readshare.csv";
317
318              // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
319              Intent shareIntent = new Intent(Intent.ACTION_SEND);
320              shareIntent.setType(isJson ? "application/json" : "text/csv");
321              shareIntent.putExtra(Intent.EXTRA_STREAM, fileUri);
322              shareIntent.putExtra(Intent.EXTRA_SUBJECT, fileName); // ■■1■EXTRA_S
323
324              // ■■2■■■ ClipData ■■■■■■■■■■■■■■■
325              ClipData clipData = ClipData.newUri(
326                  requireContext().getContentResolver(), fileName, fileUri);
327              shareIntent.setClipData(clipData);
328
329              shareIntent.addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION);
330
331              // ■■■■■■■■■■■■■
332              Intent chooserIntent = Intent.createChooser(shareIntent, "■■■■");
333              String myPackageName = requireContext().getPackageName();
334              chooserIntent.addFlags(Intent.FLAG_ACTIVITY_EXCLUDE_FROM_RECENTS);
335              chooserIntent.putExtra(Intent.EXTRA_EXCLUDE_COMPONENTS,
336                  new ComponentName[]{new ComponentName(myPackageName, myPackageNa
337
338              startActivity(chooserIntent);
339
340              // ■■■■■■
341              String format = isJson ? "JSON" : "CSV";
342              Snackbar.make(requireView(),
343                  format + " ■■■■■■ Documents ■■■■■■■...",
344                  Snackbar.LENGTH_SHORT).show();
345
346          } catch (Exception e) {
347              Snackbar.make(requireView(),
348                  "■■■■■" + e.getMessage(),
349                  Snackbar.LENGTH_LONG).show();
350          }
351      }
352
353      @Override
354      public void onPinStatusChanged() {
```

```
355          Log.d("HomeFragment", "onPinStatusChanged ■■■");
356          // ■■■■■■■
357          List<LinkItem> pinnedLinks = linkDao.getPinnedLinks();
358          Map<String, List<LinkItem>> groupedLinks = linkDao.getLinksGroupByDate()
359
360          Log.d("HomeFragment", "■■■■■■■: " + pinnedLinks.size());
361          adapter.setPinnedLinks(pinnedLinks);
362          adapter.setGroupedLinks(groupedLinks);
363      }
364
365      @Override
366      public void onRequestCustomIcon(LinkItem item, String url) {
367          // ■■■■■■■■■■■■■■■
368      }
369
370      @Override
371      public void onAddToSubject(LinkItem item) {
372
373      }
374
375      private void scrollToDate(RecyclerView recyclerView, String date) {
376          // ■■■■■■■■■■
377          int position = adapter.getPositionForDate(date);
378          if (position != -1) {
379              recyclerView.post(() -> {
380                  // ■■ LinearLayoutManager
381                  LinearLayoutManager layoutManager = (LinearLayoutManager) recycl
382                  if (layoutManager != null) {
383                      // ■■■■■■■■■
384                      int searchBoxHeight = binding.searchEditText.getHeight();
385                      // ■■■■■■■■■offset ■■■■■■■
386                      layoutManager.scrollToPositionWithOffset(position, searchBox
387                  }
388              });
389          }
390      }
391
392      public void onLinkRemarkUpdated(LinkItem item) {
393          // ■■■■■■■UI■■■■■■■■■
394          // ■■LinkDao■■■■■■■■■■adapter■■■■■■■■■■■■■■■■■■■■■■
395      }
396  }
  1  package person.notfresh.readingshare.util;
  2
  3  import android.content.Context;
  4  import android.content.SharedPreferences;
  5  import java.util.ArrayList;
  6  import java.util.Arrays;
  7  import java.util.LinkedHashSet;
  8  import java.util.List;
```

```
 9
10  public class RecentTagsManager {
11      private static final String PREF_NAME = "recent_tags_pref";
12      private static final String KEY_RECENT_TAGS = "recent_tags";
13      private static final String TAG_SEPARATOR = ",";
14      private static final String KEY_RECENT_TAGS_WINDOW = "recent_tags_window";
15      private static final int DEFAULT_RECENT_TAGS_WINDOW = 5;
16      private static final int MIN_RECENT_TAGS_WINDOW = 1;
17      private static final int MAX_RECENT_TAGS_WINDOW = 10;
18
19      public static int getRecentTagsWindow(Context context) {
20          SharedPreferences prefs = context.getSharedPreferences("settings", Conte
21          return prefs.getInt(KEY_RECENT_TAGS_WINDOW, DEFAULT_RECENT_TAGS_WINDOW);
22      }
23
24      public static void setRecentTagsWindow(Context context, int window) {
25          int validWindow = Math.max(MIN_RECENT_TAGS_WINDOW, Math.min(MAX_RECENT_T
26          SharedPreferences prefs = context.getSharedPreferences("settings", Conte
27          prefs.edit().putInt(KEY_RECENT_TAGS_WINDOW, validWindow).apply();
28      }
29
30      public static void addRecentTag(Context context, String tag) {
31          if (tag == null || tag.trim().isEmpty()) {
32              return;
33          }
34
35          SharedPreferences prefs = context.getSharedPreferences(PREF_NAME, Contex
36          String recentTagsStr = prefs.getString(KEY_RECENT_TAGS, "");
37
38          // ■■LinkedHashSet■■■■■■■■■■
39          LinkedHashSet<String> tagsSet = new LinkedHashSet<>();
40
41          // ■■■■■■■■■■■
42          tagsSet.add(tag.trim());
43
44          // ■■■■■■■
45          if (!recentTagsStr.isEmpty()) {
46              String[] existingTags = recentTagsStr.split(TAG_SEPARATOR);
47              tagsSet.addAll(Arrays.asList(existingTags));
48          }
49
50          // ■■■■■■■■■■■
51          List<String> tagsList = new ArrayList<>(tagsSet);
52          int window = getRecentTagsWindow(context);
53          if (tagsList.size() > window) {
54              tagsList = tagsList.subList(0, window);
55          }
56
57          // ■■■SharedPreferences
58          StringBuilder newTagsStr = new StringBuilder();
```

```
59          for (int i = 0; i < tagsList.size(); i++) {
60              if (i > 0) newTagsStr.append(TAG_SEPARATOR);
61              newTagsStr.append(tagsList.get(i));
62          }
63
64          prefs.edit().putString(KEY_RECENT_TAGS, newTagsStr.toString()).apply();
65      }
66
67      public static void addRecentTags(Context context, List<String> tags) {
68          if (tags == null || tags.isEmpty()) {
69              return;
70          }
71
72          // ■■■■■■■■■■■■■■■■■■■■■■■■
73          for (int i = tags.size() - 1; i >= 0; i--) {
74              addRecentTag(context, tags.get(i));
75          }
76      }
77
78      public static List<String> getRecentTags(Context context) {
79          SharedPreferences prefs = context.getSharedPreferences(PREF_NAME, Contex
80          String recentTagsStr = prefs.getString(KEY_RECENT_TAGS, "");
81
82          if (recentTagsStr.isEmpty()) {
83              return new ArrayList<>();
84          }
85
86          return Arrays.asList(recentTagsStr.split(TAG_SEPARATOR));
87      }
88  }
 1  package person.notfresh.readingshare.util;
 2
 3  public class StringUtil {
 4      // TODO ■■■■■■■
 5      // ■■■■■■■■■■■
 6      public static String extractTitle(String text) {
 7          // ■■ URL
 8          String[] parts = text.split("\\s+");
 9          StringBuilder title = new StringBuilder();
10
11          for (String part : parts) {
12              if (!part.startsWith("http://") && !part.startsWith("https://")) {
13                  if (title.length() > 0) {
14                      title.append(" ");
15                  }
16                  title.append(part);
17              }
18          }
19
20          String result = title.toString().trim();
```

```
21          return result.isEmpty() ? "■■■■■" : result;
22      }
23  }
```